UNITED INTERNATIONAL UNIVERSITY

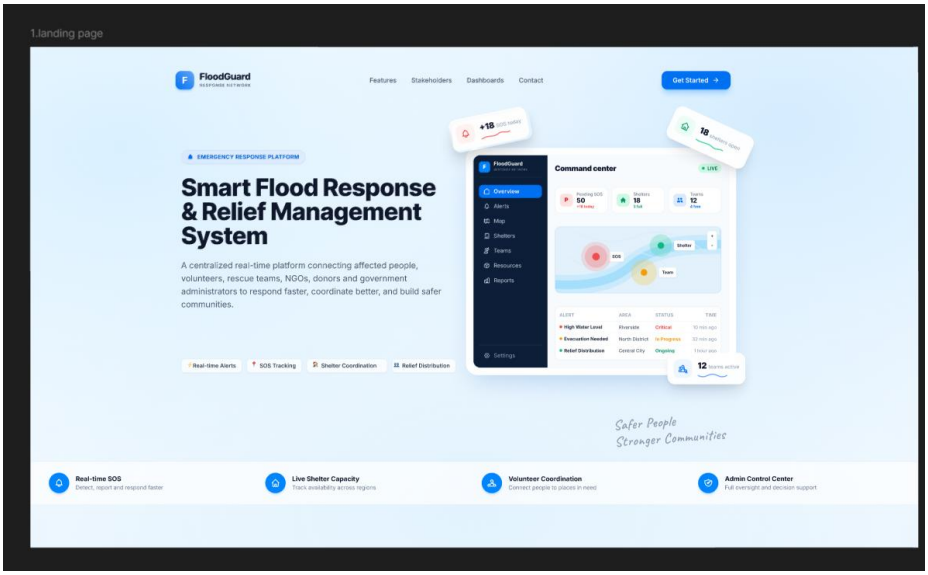
Department of Computer Science & Engineering

CSE 3412 - SYSTEM ANALYSIS AND DESIGN LAB

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Smart Flood Response & Relief Management System

FloodGuard - Response Network



FloodGuard landing page prototype

Project Team:Alpha 4

No.	Name	Student ID
1	Sumaiya Kamal Momo	0112410153
2	Tamanna Akter Omi	0112420541
3	Nafis Al Sami	0112420335
4		

Item	Information
Section	C
Group No.	8
Course	CSE 3411 - System Analysis and Design Lab
Document	Final Software Requirements Specification

Table of Contents

1. Cover Page	1
Project Team	2
2. Table of Contents	3
Abstract	4
Acknowledgement	4
3. Introduction	5
3.6 Requirements Elicitation & Analysis	7
3.7 Survey Methodology and Findings	8
3.8 Feasibility Analysis	10
4. Problem Statement	11
4.6 Benchmarking and Gap Analysis	12
4.7 Gap Analysis	13
5. Objectives	15
6. Scope	16
7. Stakeholders & Users	17
8. Requirements	18
9. System Design (Diagrams)	21
10. Database Design	28
11. UI/UX Design	31
12. System Architecture	45
13. Conclusion	48
14. Future Scope	49
15. References	50

Abstract

The Smart Flood Response & Relief Management System, branded in the prototype as FloodGuard, is a centralized web and mobile-ready platform for coordinated flood emergency response in Bangladesh. It connects affected people, volunteers, rescue teams, NGOs and relief organizations, donors, and administrators through role-specific portals. Core functions include emergency SOS with location and priority assessment, shelter discovery and capacity visibility, relief and medical requests, rescue assignment and mission tracking, volunteer coordination, NGO inventory and distribution, donation tracking, water-level and field reporting, complaints, emergency broadcasting, analytics, and administrative monitoring. The system addresses fragmented communication by creating one shared operational workflow and by supporting AI-assisted SOS prioritization, duplicate-relief alerts, and heatmap-based resource allocation. This final SRS consolidates the project problem, objectives, scope, stakeholders, functional and non-functional requirements, benchmarking and gap analysis, UML models, database design, UI/UX prototype, and infrastructure architecture into one implementation blueprint.

Acknowledgement

We acknowledge the guidance of our course instructor, the contributions of all group members, and the project artifacts produced throughout the semester. We also acknowledge the official documentation of Sahana Eden, Ushahidi, and IFRC GO used in the benchmark study. The final report integrates the initial SRS, ER design, architecture work, and FloodGuard UI prototype into a consistent system analysis and design document.

CHAPTER 3: Introduction

3.1 Purpose

This SRS defines the complete requirements and design baseline for the Smart Flood Response & Relief Management System. It explains the target problem, stakeholder roles, system behavior, data model, user interaction, architectural structure, and future development direction so that developers, testers, evaluators, and operational stakeholders can understand the proposed solution consistently.

3.2 Product Perspective

FloodGuard is designed as a centralized emergency coordination platform rather than only a reporting or warning application. Affected people and field users interact mainly through mobile-friendly interfaces, while NGOs, donors, rescue teams, and administrators use role-specific dashboards. The platform combines rescue, shelter, relief, inventory, donation, volunteer, mapping, medical support, complaints, notifications, water-level reporting, analytics, and decision-support functions.

3.3 Background

Bangladesh experiences recurring flood emergencies in which the timely movement of information is as important as the physical movement of people and resources. When SOS requests, shelter capacity, field reports, stock records, rescue assignments, and relief distributions are maintained across separate phone calls, messages, and spreadsheets, response teams may not have a shared operational picture. The project therefore focuses on connecting these activities in one system.

3.4 System Development Life Cycle (SDLC)

1. Planning - define the flood-response problem, users, boundaries, and expected outcomes.
2. Requirement gathering - identify functional services, role permissions, data needs, and emergency constraints.
3. Analysis - examine current practices, comparable systems, operational gaps, and design priorities.
4. Design - prepare UML models, database design, UI/UX prototype, and infrastructure architecture.
5. Implementation - build the portal and service modules incrementally.
6. Testing - verify functionality, security, usability, integration, performance, and recovery.
7. Deployment and maintenance - deploy with monitoring, backup, incident response, and continuous improvement.

3.5 Information Sources

Source Type	Examples	Use in This Project
Internal project sources	Initial SRS, ER analysis, architecture diagrams, UI prototype	Define project-specific requirements, entities, workflows, and interfaces.
Benchmark systems	Sahana Eden, Ushahidi, IFRC GO	Compare humanitarian logistics, reporting/mapping, and emergency information sharing.
Operational inputs	Affected people, volunteers, rescue teams, NGOs, donors, administrators	Define user needs, role responsibilities, and workflow priorities.

3.6 Requirements Elicitation & Analysis

Before designing FloodGuard, it was necessary to understand how affected persons, volunteers, rescue teams, NGOs, donors, and administrators behave and communicate during a flood emergency. Because the platform coordinates multiple stakeholders at once, requirements elicitation focused on their specific needs, responsibilities, operational pain points, and the information that must move between roles. This analysis guided decisions such as what an SOS request should capture, how shelter and relief information should be updated, how rescue status should be tracked, and what administrators need to see in a centralized dashboard.

3.6.1 Information Collected

- Affected Persons - SOS requests with location and risk details, relief demands, and shelter-search activity.
- Volunteers - registration details, shelter occupancy and local-area condition updates, and availability for coordination tasks.
- Rescue Teams - rescue status updates (accepted, in-progress, completed) and field conditions during ongoing operations.
- NGOs - relief-demand assessment, inventory and stock levels, and distribution records that support duplicate-relief checking.
- Donors - donation details and contribution information for tracking through Donation Management.
- Administrators - emergency broadcasts, complaint verification, monitoring, and system-wide coordination inputs.

3.6.2 Sources of Information

Internal sources include stakeholder requirements, intended role interactions, the initial SRS, ER analysis, architecture artifacts, and prototype designs. Proposed system interactions - such as SOS submissions, shelter searches, relief transactions, rescue status updates, water-level submissions, inventory entries, complaints, and dashboard activity - are treated as operational data sources once the platform is implemented. External sources include public geographic and flood data, mapping/geolocation services, weather or water-level APIs, SMS/notification services, relevant research and guidelines, and future expert consultation with disaster-management, rescue, NGO, and local-administration personnel.

3.7 Survey Methodology and Findings

Methodology - an online questionnaire (Google Form) was distributed to 30 respondents. Section A covered respondent background, including flood experience, frequency, recency, years in a flood-prone area, and age group. Section B covered 15 feature-specific and system-specific questions, including awareness, feature usefulness, trust in AI prioritization, notification preference, and overall adoption likelihood. Most respondents (25 of 30) were aged 18-24, so the findings primarily reflect a younger respondent pool.

3.7.1 Selected Survey Results

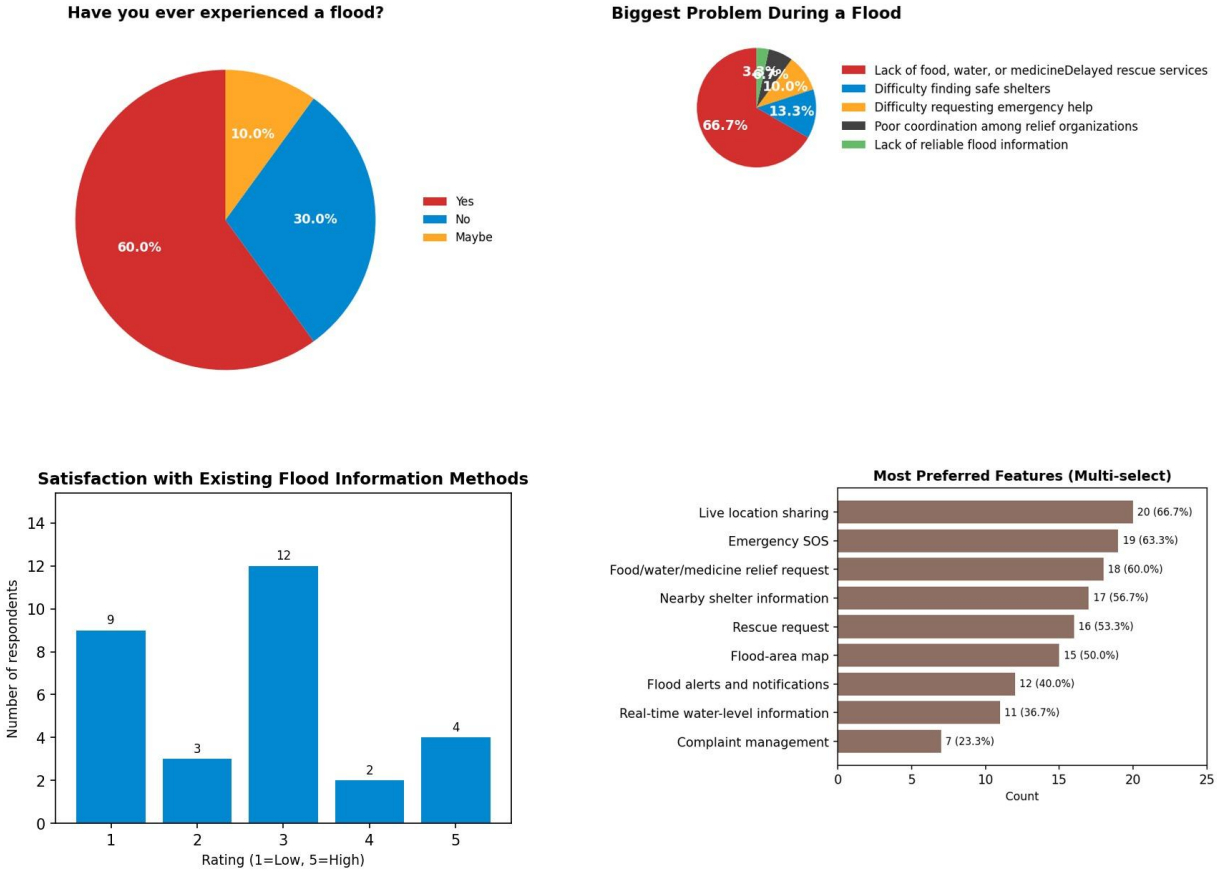


Figure 3.1: Selected survey results - flood experience, major problems, satisfaction, and preferred features

Key findings: 60.0% of respondents had experienced a flood or knew someone affected. Lack of food/water/medicine and delayed rescue together accounted for 66.7% of the reported major problems. Satisfaction with current flood-information or emergency-assistance methods was generally low to moderate. The most preferred features were live location sharing (66.7%), Emergency SOS (63.3%), food/water/medicine relief requests (60.0%), and nearby shelter information (56.7%).

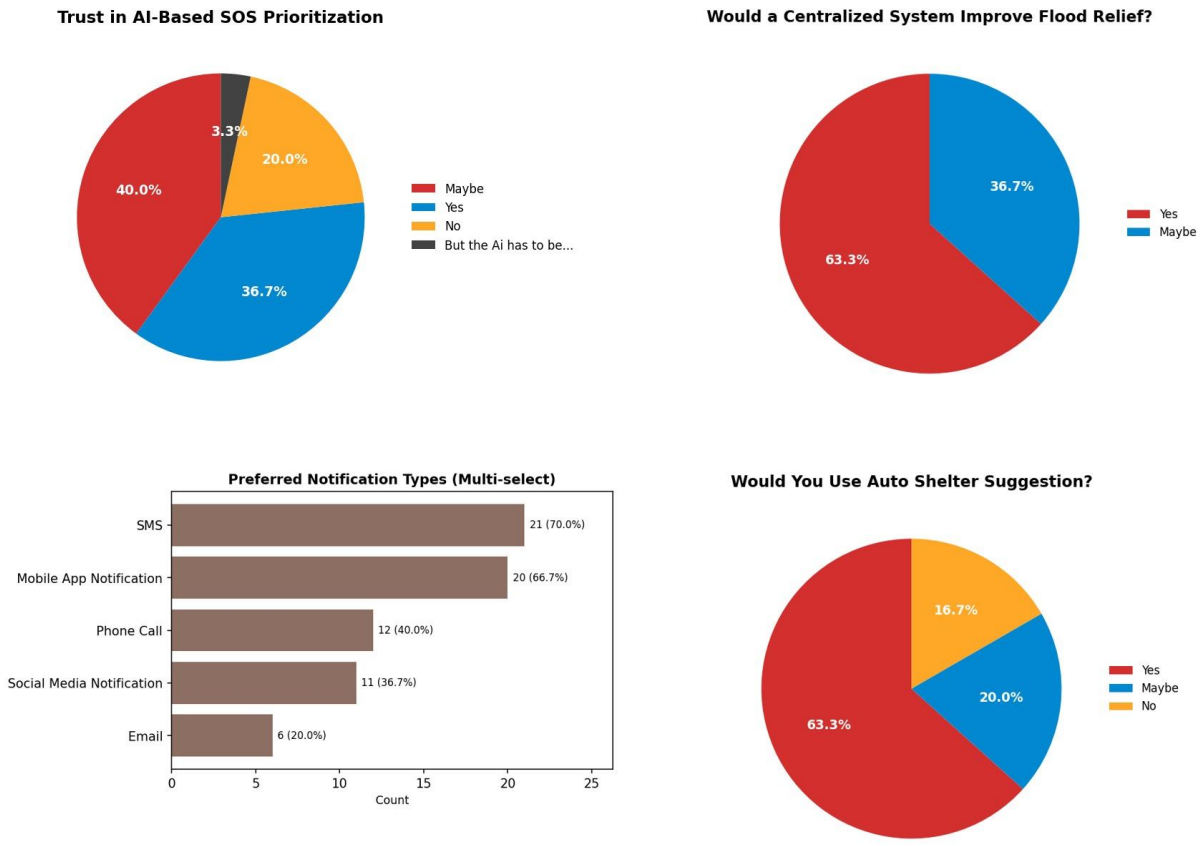


Figure 3.2: Survey results - AI trust, centralized platform value, notification preferences, and shelter suggestion

AI-based prioritization received conditional trust: 36.7% answered Yes, 40.0% Maybe, and 20.0% No. A centralized system connecting all stakeholders was supported by 63.3% of respondents, with the remaining 36.7% answering Maybe and no respondent answering No. SMS (70.0%) and mobile-app notifications (66.7%) were the most preferred alert channels. Automatic safe-shelter suggestion was supported by 63.3% of respondents.

3.7.2 Recommendations from Survey

- Prioritize live location sharing and Emergency SOS as core features.
- Strengthen relief and shelter functions, including automatic nearby-shelter suggestion.
- Treat AI-based SOS prioritization as decision support that requires validation, transparency, and human override.
- Use SMS and mobile-app notifications as primary alert channels, with other channels as secondary options.
- Emphasize real-time centralized information while recognizing that the survey sample is heavily concentrated in the 18-24 age group.

3.8 Feasibility Analysis

Feasibility Area	Analysis for FloodGuard
Technical	Mapping APIs, GPS/geolocation, flood or weather data services, SMS/notification services, web/mobile frameworks, databases, and cloud hosting are established technologies. The major risk is reliable integration under low-connectivity flood conditions, not technology availability.
Economic / Financial	For the academic project, development can rely on open-source libraries and free-tier or low-cost services. A real deployment would require recurring server, mapping, messaging, monitoring, and data-service costs; NGO or government partnerships could reduce the funding burden.
Operational	The system targets a coordination gap supported by survey feedback: 63.3% said a centralized platform would improve flood-relief management. Adoption depends on volunteers, rescue teams, NGOs, donors, and administrators using the platform consistently and keeping operational data updated.
Human Resources	The current scope can be developed by a student software team using standard web/app, database, UI/UX, and API-integration skills. Disaster-management, rescue, NGO, and local-administration experts are recommended for later workflow validation.
Market / Legal / Political	The analysis indicates room for an integrated multi-stakeholder platform, but future deployment must account for government-backed services, organizational adoption incentives, and privacy obligations for location and health-related emergency data.
Schedule	An Agile iterative approach allows high-priority features such as Emergency SOS, shelter management, and rescue coordination to be implemented first, with lower-priority modules deferred if the semester schedule is constrained.

Overall, the project is feasible as an academic system-design and implementation project, provided that the team treats connectivity, third-party integration, data privacy, AI validation, and stakeholder adoption as explicit risks rather than assumptions.

CHAPTER 4: Problem Statement

4.1 Existing Problem

Bangladesh faces recurring floods, but relief and rescue activities are often managed through manual and poorly coordinated communication. Affected people may not know which shelters have free space, how to send their exact location, whether a rescue team is coming, or when relief will arrive. At the same time, volunteers, NGOs, rescue teams, and administrators may use different phone calls, messages, lists, and spreadsheets. This fragmented process can cause delayed rescue, uneven relief distribution, repeated support in some locations, shortages in other locations, and weak visibility of the overall flood situation.

4.2 Why the Problem Is Important

Flood emergencies are time-sensitive. A delayed SOS response can directly affect human safety. Incorrect shelter or stock information can leave families without safe accommodation, food, water, medicine, or transport. Poor coordination also wastes limited relief resources. Therefore, the problem affects response speed, fairness, transparency, and decision-making.

4.3 Who Is Affected

- Affected people and families who need rescue, shelter, relief, medical support, and trusted alerts.
- Rescue teams that need accurate victim location, urgency, and case-status information.
- Volunteers who need clear assignments and an easy way to report local conditions.
- NGOs and relief organizations that manage demand, inventory, donations, volunteers, and distribution.
- Donors who need confidence that resources are used properly.
- Administrators or UNO offices that need a complete view for resource allocation, monitoring, emergency broadcasts, and reporting.

4.4 Why a Software Solution Is Necessary

A centralized software platform can connect all these users with shared and continuously updated information. SOS requests can include location and emergency details. Rescue teams can receive prioritized cases and update rescue status. Shelters can publish capacity. NGOs can compare relief demand with available stock and previous distributions. Administrators can monitor dashboards, maps, and heatmaps. Users can receive emergency alerts. The software also supports functions that are difficult to manage accurately through manual coordination, such as AI-assisted SOS priority prediction, duplicate-relief alerts, integrated complaint tracking, and cross-module reports.

4.5 Motivation for the Proposed System

The project is motivated by the need for a Bangladesh-focused flood response platform where rescue, shelter, relief, donations, volunteers, water-level reporting, complaints, and administrative decision support work as one connected process. The goal is not to replace weather forecasting or physical rescue services. The goal is to make existing people, information, and resources easier to coordinate.

4.6 Benchmarking and Gap Analysis

4.6.1 Benchmark Analysis

Three existing systems were reviewed because they support important parts of disaster response, crisis reporting, humanitarian logistics, or emergency information management: Sahana Eden, Ushahidi, and IFRC GO. The purpose of the benchmark is to understand their useful features and identify the areas that are not fully covered for the proposed Bangladesh flood-response workflow.

4.6.1.1 Sahana Eden

Sahana Eden is an open-source humanitarian and disaster-management platform. Official Sahana materials describe support for organizations, volunteers, inventory, requests, donations, shelters, assessments, mapping, events, and messaging.

Main Features	Strengths	Weaknesses / Limitations
Organization registry; volunteer and HR management; inventory, requests, donations, warehouses and shipments; shelter registry; mapping and situation awareness; configurable messaging.	Strong humanitarian feature set; open-source and configurable; good coordination among organizations, facilities, people, and resources.	Presented by Sahana Foundation as part of a legacy archive; reviewed material does not present AI-based SOS priority, duplicate-relief alerts, or heatmap-based resource allocation as core end-user features; broad humanitarian scope may be heavier than the focused semester workflow.

4.6.1.2 Ushahidi

Ushahidi is a crowdsourcing and mapping platform for collecting, managing, visualizing, and analyzing reports from communities. Its official feature pages emphasize surveys, location-based submissions, mobile support, filters, workflows, role-based permissions, alerts, and map-based analysis.

Main Features	Strengths	Weaknesses / Limitations
Crowdsourced reports and surveys; location-based submissions; map/data/activity views; filters and workflows; permissions; email/SMS alerts; mobile reporting and export.	Very strong for community reporting, incident mapping, rapid data collection, and geographical visualization; flexible for flood reports; supports collaboration and notifications.	General crowdsourcing platform rather than a complete relief-operations system; lacks dedicated shelter-capacity, inventory/distribution, donation, rescue-team lifecycle, and medical/ambulance modules as built-in core modules; follow-through from SOS to rescue completion is less central.

4.6.1.3 IFRC GO

IFRC GO is the International Federation of Red Cross and Red Crescent Societies emergency operations platform. IFRC describes GO as a platform for capturing, analyzing, and sharing real-time crisis data and operational information.

Main Features	Strengths	Weaknesses / Limitations
Real-time field reports and operational updates; maps, graphs, dashboards, public emergency information; coordination across the Red Cross/Red Crescent network; 3W coordination.	Strong institutional emergency information management; shared operational picture using reports, maps, dashboards and field information; useful for coordination and analysis across response organizations.	Primary focus is organizational emergency operations and information sharing; not a local citizen-facing workflow combining SOS tracking, shelter search, relief demand/distribution, complaints, donations, and rescue completion in one application.

4.6.2 Comparative Feature Analysis

Capability	Sahana Eden	Ushahidi	IFRC GO	Proposed System

Citizen reporting	Partial	Strong	Partial	Strong
Location mapping	Strong	Strong	Strong	Strong
Shelter management	Strong	Partial	Partial	Strong
Rescue task status	Partial	Partial	Partial	Strong
Relief workflow	Strong	Partial	Partial	Strong
Inventory / stock	Strong	Not indicated	Not indicated	Strong
Donation tracking	Strong	Not indicated	Partial	Strong
Volunteer coordination	Strong	Partial	Partial	Strong
Medical / ambulance	Partial	Not indicated	Partial	Strong
Water-level reports	Partial	Strong for reporting	Partial	Strong
Role-based access	Strong	Strong	Strong	Strong
Emergency alerts	Strong	Strong	Strong	Strong
AI SOS priority	Not indicated	Not indicated	Not indicated	Strong
Duplicate-relief alert	Not indicated	Not indicated	Not indicated	Strong
Heatmap allocation	Partial	Strong mapping	Partial analytics	Strong
Complaint tracking	Partial	Partial	Not indicated	Strong
Admin analytics	Strong	Strong	Strong	Strong

Overall, Sahana Eden is strongest in humanitarian logistics and shelter management, Ushahidi is strongest in crowdsourced reporting and mapping, and IFRC GO is strongest in institutional emergency information sharing and dashboards. The proposed system combines selected strengths from all three and applies them to a focused Bangladesh flood-response workflow.

4.7 Gap Analysis

The benchmark analysis shows that useful disaster-response systems already exist, but none of the three reviewed systems exactly matches the proposed semester project. The main gap is not basic mapping or reporting. The main gap is one focused workflow that directly connects a flood victim's request with rescue, shelter, relief, volunteer, medical, donation, complaint, and administrative activities.

Identified Gap	Benchmark Observation	Proposed Response
Fragmented operational flow	Existing platforms specialize in different areas: Sahana in humanitarian modules, Ushahidi in crowdsourced reporting and mapping, and IFRC GO in institutional information sharing.	Connect SOS, rescue, shelters, relief, inventory, volunteers, donations, medical support, complaints, alerts, and admin monitoring in one flood-focused workflow.
SOS prioritization	The reviewed official benchmark material does not identify AI-based victim-risk priority ranking as a core feature.	Use AI-assisted priority prediction based on risk factors such as age, illness, children, pregnant women, and water level, while keeping human control.
Duplicate and imbalanced relief	Logistics systems can manage requests and inventory, but duplicate-relief detection is not highlighted in the reviewed benchmarks.	Generate duplicate-relief alerts and combine demand, stock, distribution, and heatmap information to reduce oversupply and shortages.

Citizen-to-responder visibility	Crowdsourcing systems collect reports, but end-to-end rescue completion tracking is less central.	Allow victims to submit SOS and location, rescue teams to accept and update cases, and administrators to monitor progress.
Shelter, relief, and medical connection	Existing systems may support some parts separately, but not the exact integrated user flow.	Link shelter availability, rescue needs, relief demand, medical/ambulance requests, and broadcast alerts through shared users and dashboards.
Accountability and grievance handling	Complaint resolution is not a prominent core feature in the reviewed benchmark pages.	Provide complaint submission and resolution-status tracking with distribution and administrative records.

4.8 How the Proposed System Is Different

- Flood-specific end-to-end coordination: follows the sequence from requesting help and locating victims to rescue, shelter, relief, medical support, monitoring, and reporting.
- Multi-stakeholder role model: affected people, volunteers, rescue teams, NGOs, donors, and administrators have different permissions and operational views.
- Decision support beyond basic mapping: AI-based SOS priority prediction, duplicate-relief alerts, and heatmap-based resource allocation support faster and more balanced decisions.
- Transparency and accountability: donation use, distribution records, complaint status, dashboards, reports, and monitoring improve operational visibility.
- Designed for emergency constraints: usability, low-bandwidth performance, availability, security, privacy, reliability, data integrity, backup/recovery, and accurate location handling are emphasized.

4.9 Justification of the Proposed Solution

The proposed system should not be described as universally better than mature platforms such as Sahana Eden, Ushahidi, or IFRC GO. It is better aligned with the specific academic problem and the target flood-response workflow. It combines the capabilities most relevant to this project, including citizen SOS, rescue coordination, shelter information, relief demand and distribution, inventory, volunteer and donation management, water-level reporting, alerts, complaints, and administration, and then adds project-specific decision-support features. Sahana demonstrates structured humanitarian modules and logistics; Ushahidi demonstrates simple community reporting and mapping; IFRC GO demonstrates a shared operational picture, real-time field information, and dashboards. FloodGuard uses these lessons while keeping its own scope and requirements.

CHAPTER 5: Objectives

- Analyze the existing flood relief, rescue, shelter, and emergency coordination process.
- Identify functional and non-functional requirements of flood-response stakeholders.
- Design an efficient web/mobile-based platform for shelter, relief, rescue, donation, volunteer, and reporting management.
- Allow affected people to send SOS requests, share location, find shelters, request relief and medical support, submit complaints, and receive alerts.
- Help rescue teams respond faster through location-aware SOS queues, status tracking, and priority ranking.
- Reduce duplicate and imbalanced relief distribution through demand tracking, stock monitoring, distribution records, and duplicate-relief alerts.
- Support NGOs and administrators with heatmap-based resource allocation, water-level monitoring, emergency broadcasts, dashboards, reports, and analytics.
- Maintain role-based access, privacy, data integrity, reliability, availability, and recovery for emergency data.

CHAPTER 6: Scope

6.1 In Scope

- User registration, login, profile management, and role-based access control.
- Flood-area mapping, shelter search, capacity, occupancy, and live status updates.
- SOS/emergency requests with GPS location, rescue assignment, live status, and ambulance/medical support.
- Relief demand, distribution tracking, donation management, inventory/stock monitoring, and duplicate-relief alerts.
- Volunteer registration, task coordination, flood situation reporting, water-level monitoring, and complaint/grievance management.
- AI-based SOS priority prediction, heatmap-based resource allocation, admin dashboard, reporting, analytics, and emergency broadcast alerts.

6.2 Out of Scope

- Payment gateway integration.
- Development of new IoT sensors or drone hardware.
- Flood forecasting or weather modeling.
- Fully offline SMS-based operation.
- International relief coordination.

6.3 Assumptions and Dependencies

- Users have mobile data, Wi-Fi, or another internet connection; low-bandwidth usability is required but the system is not fully offline.
- Rescue teams, NGOs, volunteers, and administrators keep shelter, inventory, rescue, and distribution information reasonably updated.
- Location information is available when users allow sharing and device/network conditions support it.
- External integrations depend on availability and permission of third-party services.
- Sensitive personal, emergency, and location data are visible only to authorized roles.

CHAPTER 7: Stakeholders & Users

Stakeholder / User	Primary Interaction	Key Need
Affected Person / Victim	Sends SOS, searches shelter, requests relief/medical support, receives alerts, reports local conditions, submits complaints.	Simple and fast emergency actions, especially on low bandwidth.
Volunteer	Reports water level/shortages, updates shelters, assists relief distribution, receives field tasks.	Quick data entry and clear task assignment.
Rescue Team	Receives prioritized SOS, views victim location, accepts missions, updates mission status and closes cases.	Reliable location, priority information, and fast status updates.
NGO / Relief Organization	Manages demand, inventory, donations, volunteers, distributions, grievances and reports.	Operational dashboard, stock visibility, and coordination.
Donor	Provides food, water, medicine, shelter items, or other resources and views utilization.	Transparency about where donations are needed and used.
Administrator / UNO Office	Monitors flood situation, manages shelters/resources, rescue teams, broadcasts, heatmaps, users/roles, and analytics.	Cross-module visibility, permissions, accountability and decision support.

CHAPTER 8: Requirements

8.1 Functional Requirements

ID	Requirement	Description
FR-01	User Registration and Login	Users shall register, sign in, and manage accounts based on assigned roles.
FR-02	Role-Based Access Control	The system shall provide permissions and features according to user role.
FR-03	Flood Area Mapping	Administrators shall mark flood-affected areas and users shall view affected zones on an interactive map.
FR-04	Shelter Management	The system shall display shelter locations, capacity, details, occupancy, and availability updates.
FR-05	Emergency SOS Request	Affected people shall send SOS requests, share location, and track request status.
FR-06	Emergency Service Request	Users shall submit rescue, medical, and ambulance requests.
FR-07	Rescue Team Management	Administrators shall assign rescue teams; rescue teams shall update operational status and location.
FR-08	Relief Demand Management	Authorized users shall register area-wise demand for food, water, medicine, and other relief items.
FR-09	Relief Distribution Management	NGOs and administrators shall record, manage, and track relief distribution by area.
FR-10	Medical and Ambulance Services	The system shall support medical assistance and ambulance request coordination.
FR-11	Volunteer Registration and Coordination	Volunteers shall register and authorized users shall assign tasks/areas.
FR-12	Donation Management	The system shall record donations and track distribution/use of donated resources.
FR-13	Inventory and Stock Management	NGOs and administrators shall record, verify, and monitor available relief stock.
FR-14	AI-Based Priority Prediction	The system shall assign priority scores to SOS requests using risk factors and support human review.
FR-15	Duplicate Relief Alert	The system shall check previous relief distribution records and alert when duplicate/overlapping relief is detected.
FR-16	Emergency Broadcast and Alerts	Administrators shall send area-based emergency announcements and alerts.
FR-17	Water Level Monitoring	Authorized users shall record water-level updates and display area-wise information.
FR-18	Heatmap-Based Resource Allocation	The system shall generate heatmaps from SOS, demand, shelter and water-level data to identify urgent needs.
FR-19	Flood Situation Reporting	Users and volunteers shall report flood conditions, damage, affected locations and shortages.
FR-20	Complaint and Grievance Management	Users shall submit complaints and track resolution status.
FR-21	Admin Dashboard	Administrators shall monitor flood situations, shelters, SOS, rescue, relief, inventory, donations, complaints and alerts.
FR-22	Reporting and Analytics	The system shall generate reports and analytics on flood damage, shelter occupancy, relief distribution, rescue activities, donations and historical trends.

8.2 Non-Functional Requirements

ID	Quality Attribute	Description
NFR-01	Usability	Simple, clear interfaces for users with limited technical knowledge.
NFR-02	Performance	Fast response and usable behavior under low-bandwidth conditions.
NFR-03	Security	Secure authentication, protected passwords, RBAC and controlled access.
NFR-04	Availability	Remain available during flood emergencies and high demand.
NFR-05	Reliability	Process and store critical SOS, shelter, relief and rescue data without unnecessary loss.
NFR-06	Scalability	Support growing numbers of users, shelters, NGOs, teams and requests.
NFR-07	Data Integrity	Maintain accurate and consistent records and reduce duplicate/conflicting data.
NFR-08	Maintainability	Support efficient updates, maintenance and improvement.
NFR-09	Compatibility	Work on common web browsers and desktop/mobile devices.
NFR-10	Accessibility	Emergency functions should be easy to find and use.
NFR-11	Location Accuracy	Location data should be sufficiently accurate for emergency response.
NFR-12	Backup and Recovery	Important system data shall be backed up and recoverable after failure.
NFR-13	Privacy	Personal, emergency and location data shall be protected and role-limited.
NFR-14	Interoperability	Support integration with maps, SMS/email, hospital/ambulance and official data sources when required.

CHAPTER 9: System Design (Diagrams)

This chapter translates the approved requirements and analysis into system-design views showing external interactions, stakeholder functionality, internal data flow, emergency process logic, responsibility separation, domain structure, and SOS interaction sequencing. The diagrams below are the system-design artifacts adopted for the final FloodGuard SRS.

9.1 Context (Data Flow) Diagram

The context diagram represents the Smart Flood Response & Relief Management System as a single process and shows the six external stakeholder entities and the direction of the major information flows between them.

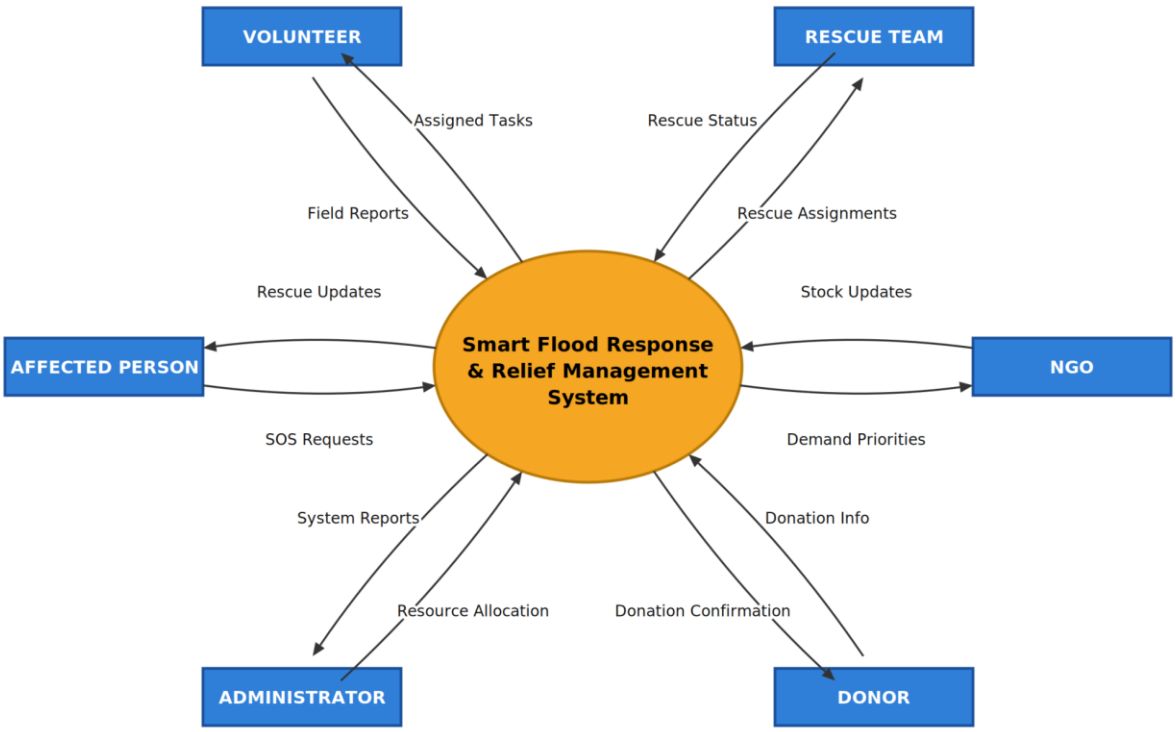


Figure 9.1: Context Diagram - Smart Flood Response & Relief Management System

9.2 Use-Case Diagram

The use-case diagram describes the system's functionality from the perspective of Affected Persons, Volunteers, Rescue Teams, NGOs, Donors, and Administrators, including the core actions each stakeholder can perform.

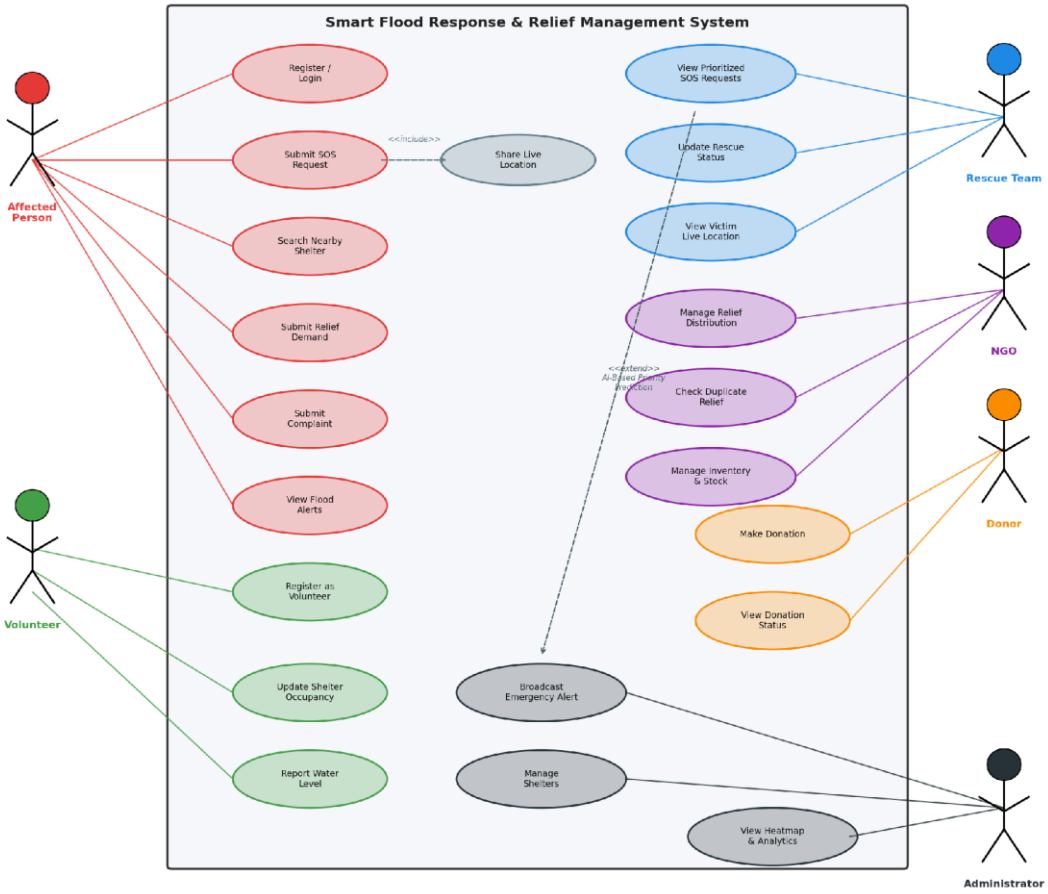


Figure 9.2: Use-Case Diagram - Smart Flood Response & Relief Management System

9.2.1 Use Case Description: Submit Emergency SOS Request (UC-01)

This use case describes how an Affected Person sends an emergency SOS request with their location during a flood, after which the request is prioritized and forwarded to the appropriate Rescue Team for action.

Users and Stakeholders

- Affected Person: wants fast rescue assistance during an emergency.
- Rescue Team: needs accurate location and risk details to respond quickly.
- Administrator: needs visibility into all active SOS requests and escalation cases.

Primary Actor: Affected Person

Pre-Conditions

- The user must be logged in to the system.
- Location access should be enabled on the user's device; manual location entry is available as a fallback.

Success Scenario

- The user taps the Emergency SOS button.
- The system captures the user's live location.
- The user provides basic risk details, such as number of people and medical needs.
- The system submits the SOS request and assigns a priority level.
- The Rescue Team receives the request and accepts it.
- The user receives confirmation and status updates.

Alternative Scenario

- If GPS is unavailable, the user manually enters the location.
- If no nearby Rescue Team is available, the request is escalated to the Administrator for reassignment.

Post-Conditions

- The SOS request is stored in the system.
- The assigned Rescue Team is notified.
- The user receives status updates until the request is resolved.

9.3 Data Flow Diagram (Level 1)

The Level 1 DFD decomposes the platform into its main processes and shows how data moves between the six stakeholder entities, operational processes, and underlying data stores.

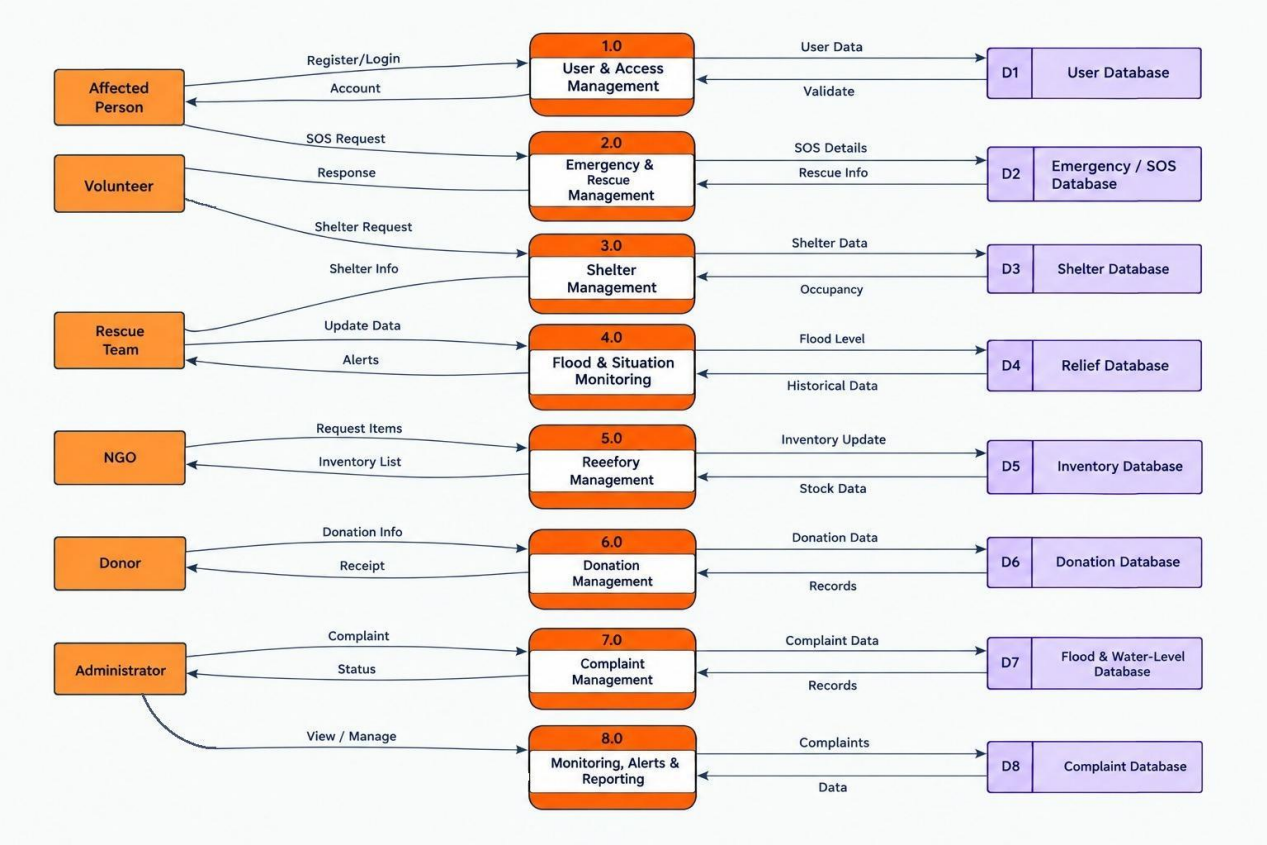


Figure 9.3: Level 1 Data Flow Diagram - Smart Flood Response & Relief Management System

9.4 Activity Diagram - Emergency SOS Request Flow

The activity diagram models the SOS process from login and location capture through risk submission, rescue-team availability checking, escalation when necessary, acceptance, confirmation, and status updates.

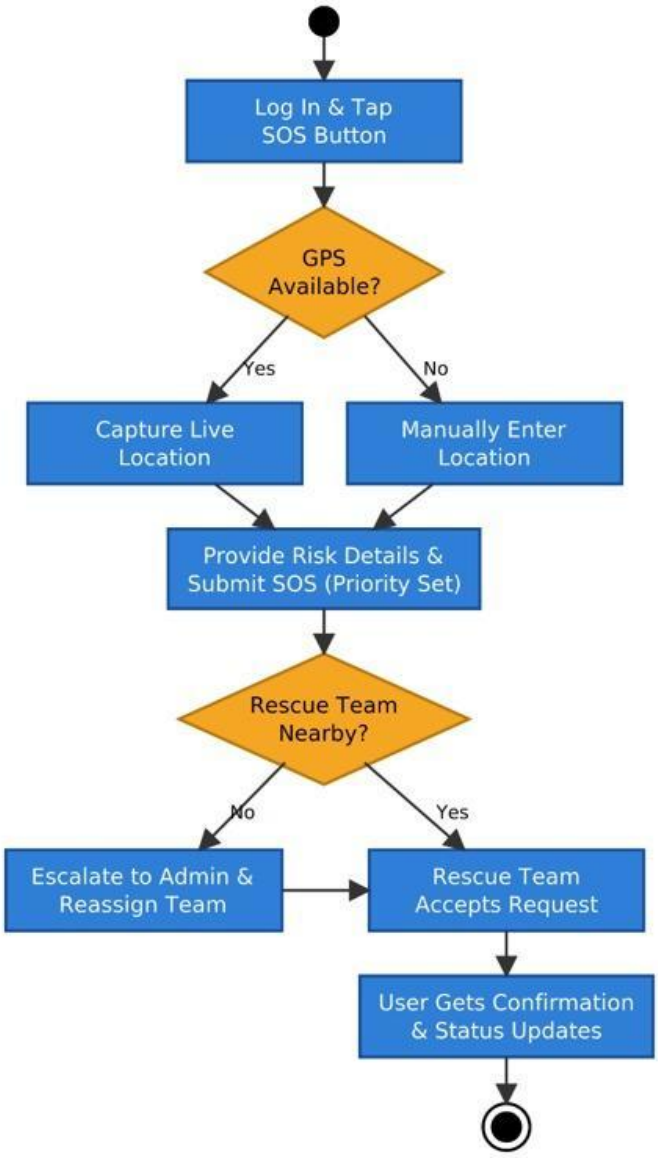


Figure 9.4: Activity Diagram - Emergency SOS Request Flow

9.5 Swimlane Diagram - Emergency SOS Request Flow

The swimlane diagram separates the Emergency SOS workflow across the Affected Person, System, Rescue Team, and Administrator so responsibility for each action and decision is explicit.

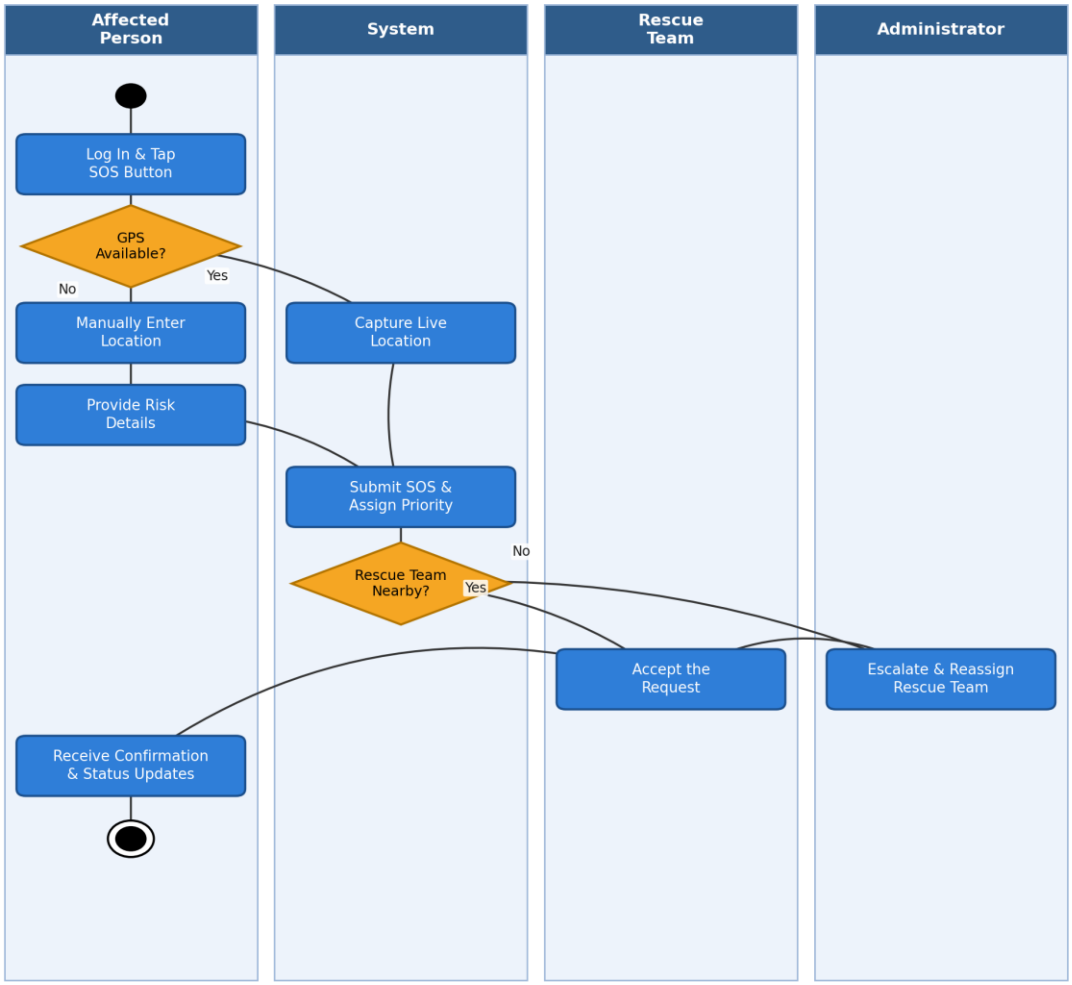


Figure 9.5: Swimlane Diagram - Emergency SOS Request Flow

9.6 Class Diagram

The class diagram illustrates the principal domain classes and their relationships, including AffectedPerson, Volunteer, RescueTeam, NGO, Donor, Administrator, SOSRequest, Shelter, ReliefDemand, Donation, and Complaint.

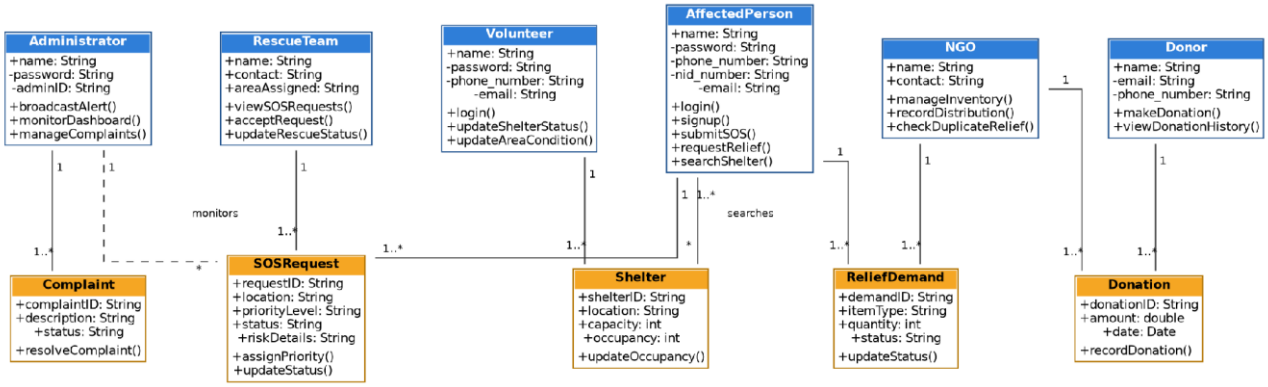


Figure 9.6: Class Diagram - Smart Flood Response & Relief Management System

9.7 Sequence Diagram - Emergency SOS Request Flow

The sequence diagram shows the time-ordered interaction when an Affected Person submits an Emergency SOS request. The request passes through the Flood Response System to the Rescue Team, while the Administrator is notified for oversight.

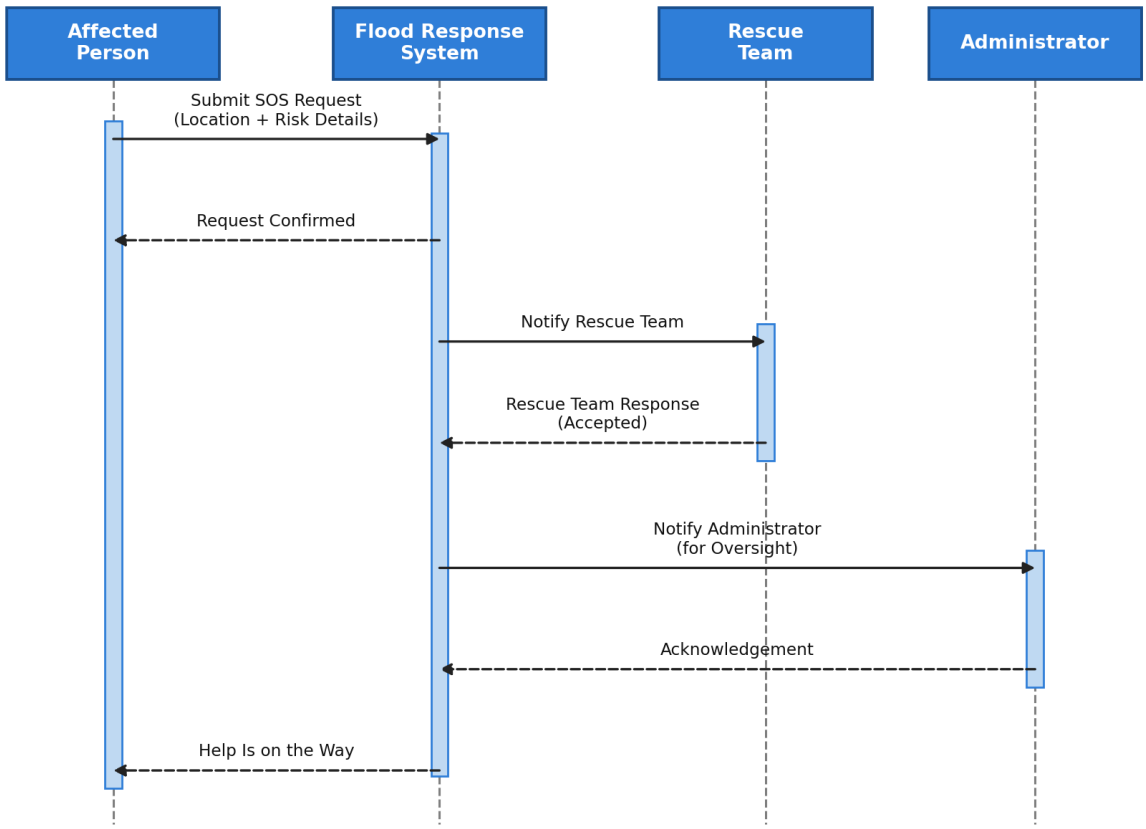


Figure 9.7: Sequence Diagram - Emergency SOS Request Flow

CHAPTER 10: Database Design

10.1 ER Design Context

The database stores both master data, such as users, roles, shelters, NGOs, rescue teams and flood areas, and transaction data, such as SOS requests, rescue assignments, relief distributions, donations, complaints, notifications and reports. The ER model contains 19 entities and uses primary and foreign keys to preserve relational integrity.

10.2 Entity Summary

Entity	Purpose
ROLE	System roles and access categories.
USER	Common account/profile information.
SOS_REQUEST	Emergency requests with location, priority and status.
RESCUE_ASSIGNMENT	Links SOS requests to rescue teams and mission history.

RESCUE_TEAM	Team identity, contact, members and availability.
VOLUNTEER	Volunteer-specific profile linked to a user.
VOLUNTEER_TASK	Volunteer task assignment and status.
COMPLAINT	User complaints and grievance status.
DONATION	Donation transaction and status.
SHELTER	Shelter location, capacity, space and operating status.
SHELTER_ALLOCATION	User-to-shelter allocation with check-in/out.
RELIEF_DEMAND	Requests for food, water, medicine or other items.
RELIEF_DISTRIBUTION	Actual distribution against demand and inventory.
INVENTORY	NGO-maintained relief stock.
NGO	Relief organization information.
FLOOD_AREA	Mapped flood-affected area and risk data.
WATER_LEVEL_RECORD	Time-based water-level observations.
NOTIFICATION	Alerts/messages sent to users.
REPORT	Generated system reports and file paths.

10.3 Entity-Relationship Diagram

The following ER diagram is the project database design artifact and shows the primary keys, foreign keys, entity attributes, and one-to-many relationships.

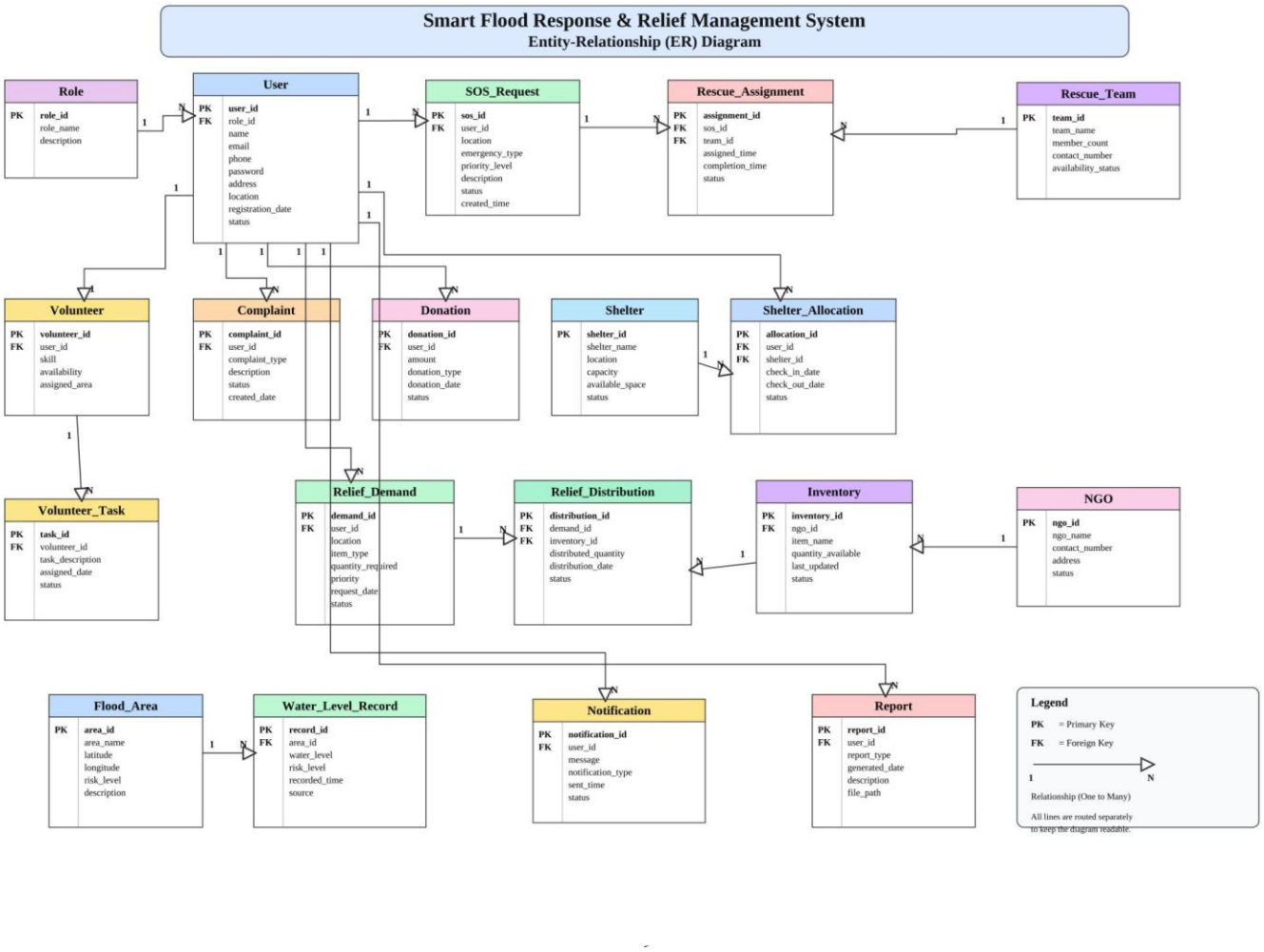


Figure 10.1: Entity-Relationship diagram

10.4 Key Relationship Logic

- ROLE to USER is 1:N; one role can be assigned to many users.
- USER to SOS_REQUEST is 1:N; one user can submit multiple SOS requests over time.
- SOS_REQUEST and RESCUE_TEAM are connected through RESCUE_ASSIGNMENT to retain assignment history.
- USER and SHELTER are connected through SHELTER_ALLOCATION so users can have allocation history and shelters can host many users.
- RELIEF_DEMAND and INVENTORY are connected through RELIEF_DISTRIBUTION so partial or multiple distributions can be recorded.
- NGO to INVENTORY is 1:N, keeping stock ownership identifiable.
- FLOOD_AREA to WATER_LEVEL_RECORD is 1:N, preserving time-series water observations.

CHAPTER 11: UI/UX Design

11.1 Introduction

The FloodGuard prototype follows a calm emergency-response visual language: light blue and white content surfaces, dark navy portal navigation, strong blue primary actions, compact status chips, map-centered dashboards, and high-visibility red/orange emergency states. The interfaces are role-specific so users see only the functions relevant to their responsibilities.

11.2 Prototyping Tool and Design Approach

- Figma is used for high-fidelity portal designs and interaction planning.
- Dashboard layouts prioritize situation awareness, current status, map context, and immediate actions.
- Critical actions such as Send Emergency SOS and Broadcast Alert use clear placement and high-contrast buttons.
- Repeated navigation patterns reduce learning effort across stakeholder portals.

11.3 UI Design Principles

Principle	Application in FloodGuard
Consistency	All portals use the same sidebar structure, card language, status chips, spacing and map conventions.
Visibility of status	LIVE indicators, alert banners, counts, progress and status colors provide continuous feedback.
Recognition over recall	Actions such as SOS, shelter, relief, medical help, inventory and broadcast remain visible in navigation.
Error prevention	Forms expose required fields, verified status, constraints and confirmation-oriented primary actions.
User control	Users can save drafts, review requests, return to role selection, navigate to settings and inspect live state.
Accessibility	Readable contrast, large emergency actions, simple wording, and responsive layouts support stressed users.

11.4 Visual System

- Primary palette: light blue and white surfaces, dark navy navigation, vivid blue primary actions.
- Status palette: red for severe/urgent, orange for warning, green for verified/available/completed, purple for analytics/resolution indicators.
- Typography: bold modern sans-serif headings with restrained body text for scanability.
- Layout: card-based dashboards, persistent left navigation, map and decision panels, compact KPI cards, clear white space.

11.5 Landing Page

The landing page introduces FloodGuard as a Smart Flood Response & Relief Management System and highlights real-time alerts, SOS tracking, shelter coordination, relief distribution, volunteer coordination, and the admin command center.

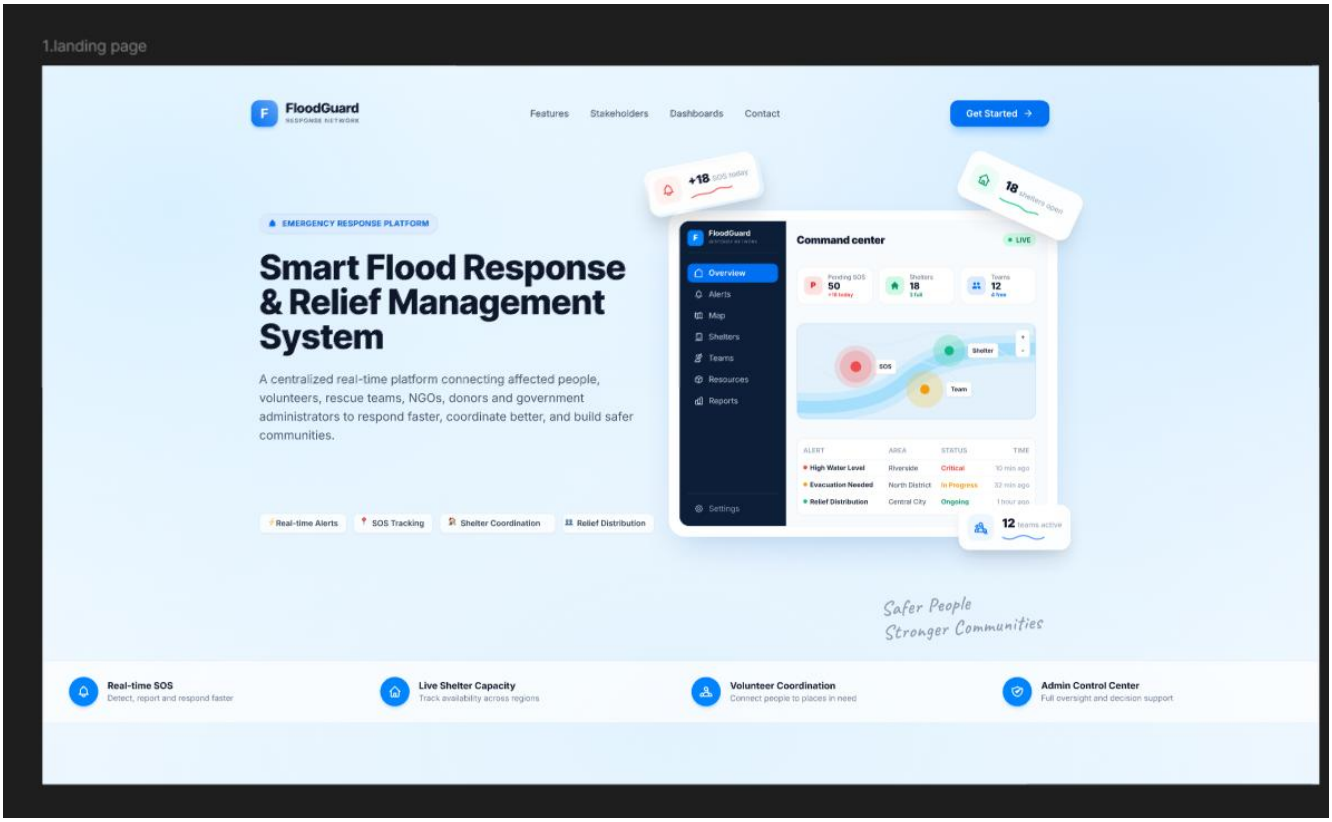


Figure 11.5: Landing Page prototype

11.6 Authentication and Volunteer Registration

The sign-in interface uses a split layout with emergency-response branding, a map illustration, secure credentials, remember-device support, password recovery, and a single strong sign-in action.

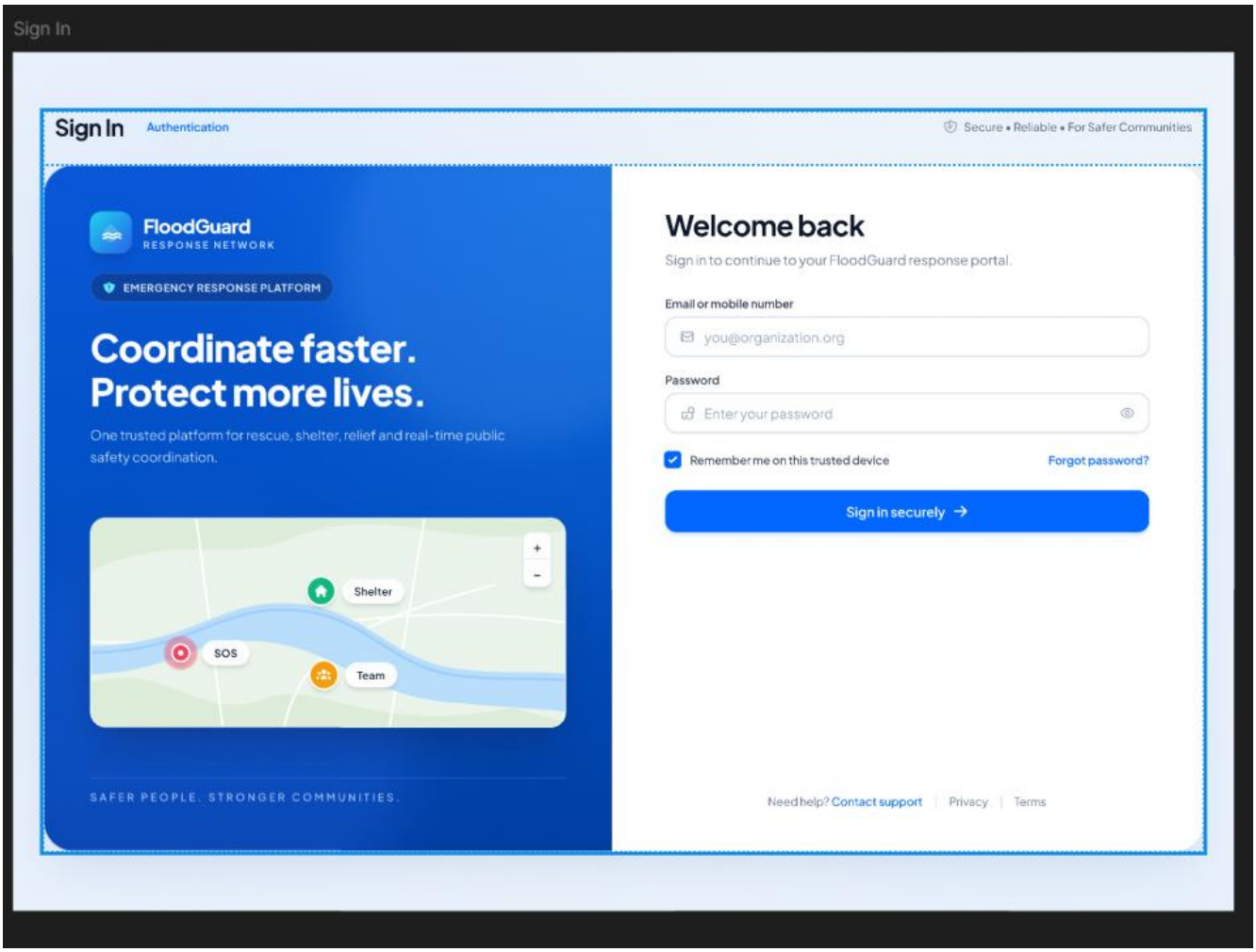


Figure 11.6: Authentication and Volunteer Registration prototype

11.7 Volunteer Registration

Volunteer registration collects identity, location, skills/expertise, availability, organization affiliation, emergency contact, and password information while showing field-support responsibilities.

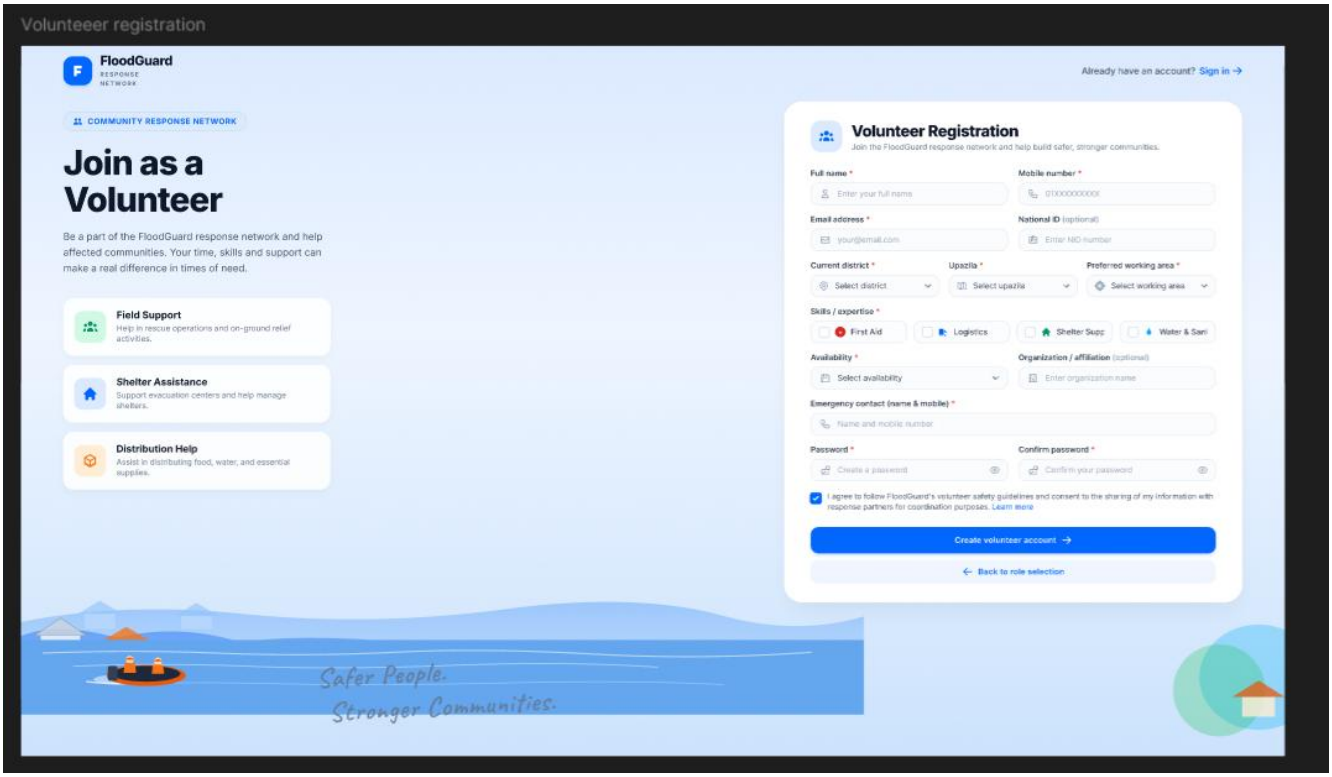


Figure 11.7: Volunteer Registration prototype

11.8 Affected Person Overview

The citizen dashboard combines warning banners, SOS/shelter/team/critical-need KPIs, a live situation map, decision queue, and recent activity.

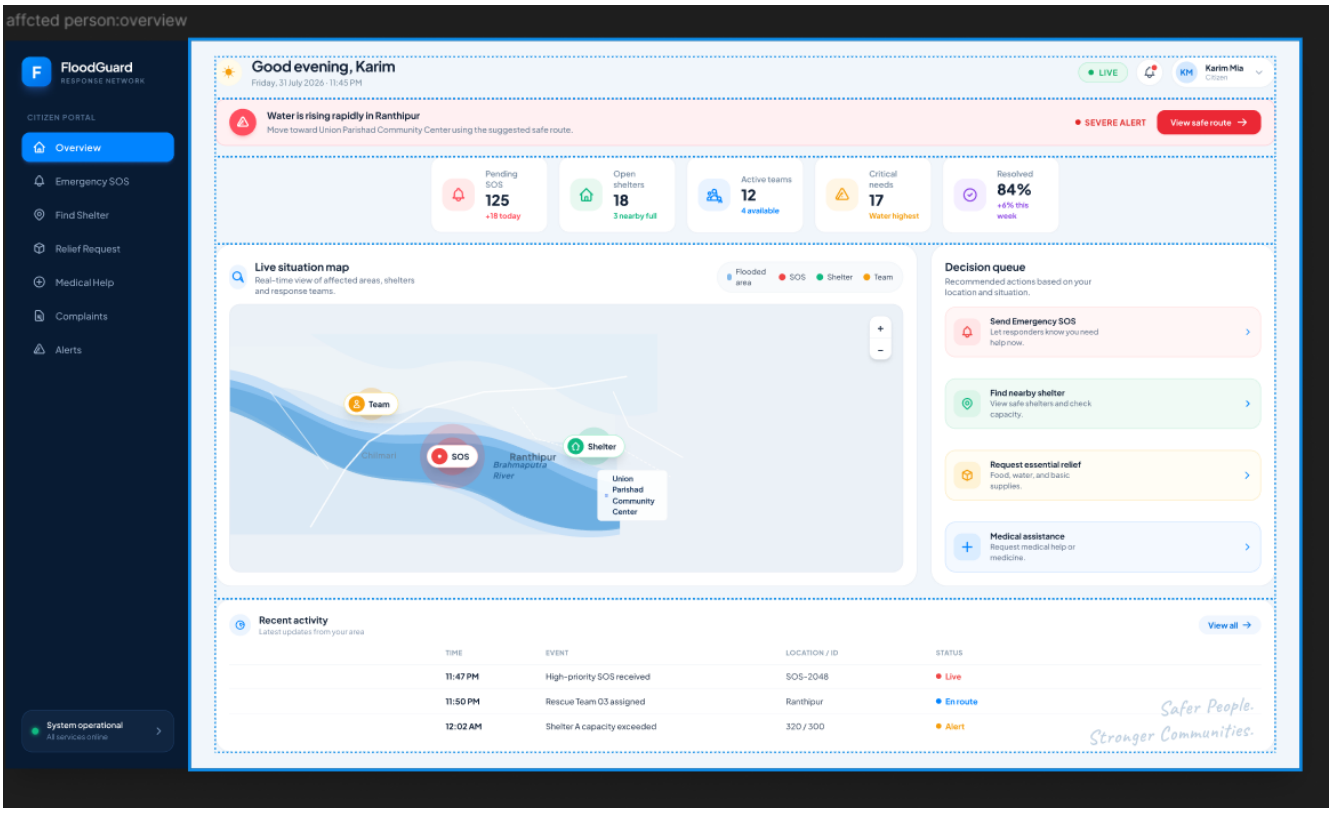


Figure 11.8: Affected Person Overview prototype

11.9 Find Safe Shelter

The shelter page prioritizes map-based discovery, nearby shelter ranking, capacity/status visibility, and a safe-directions action.

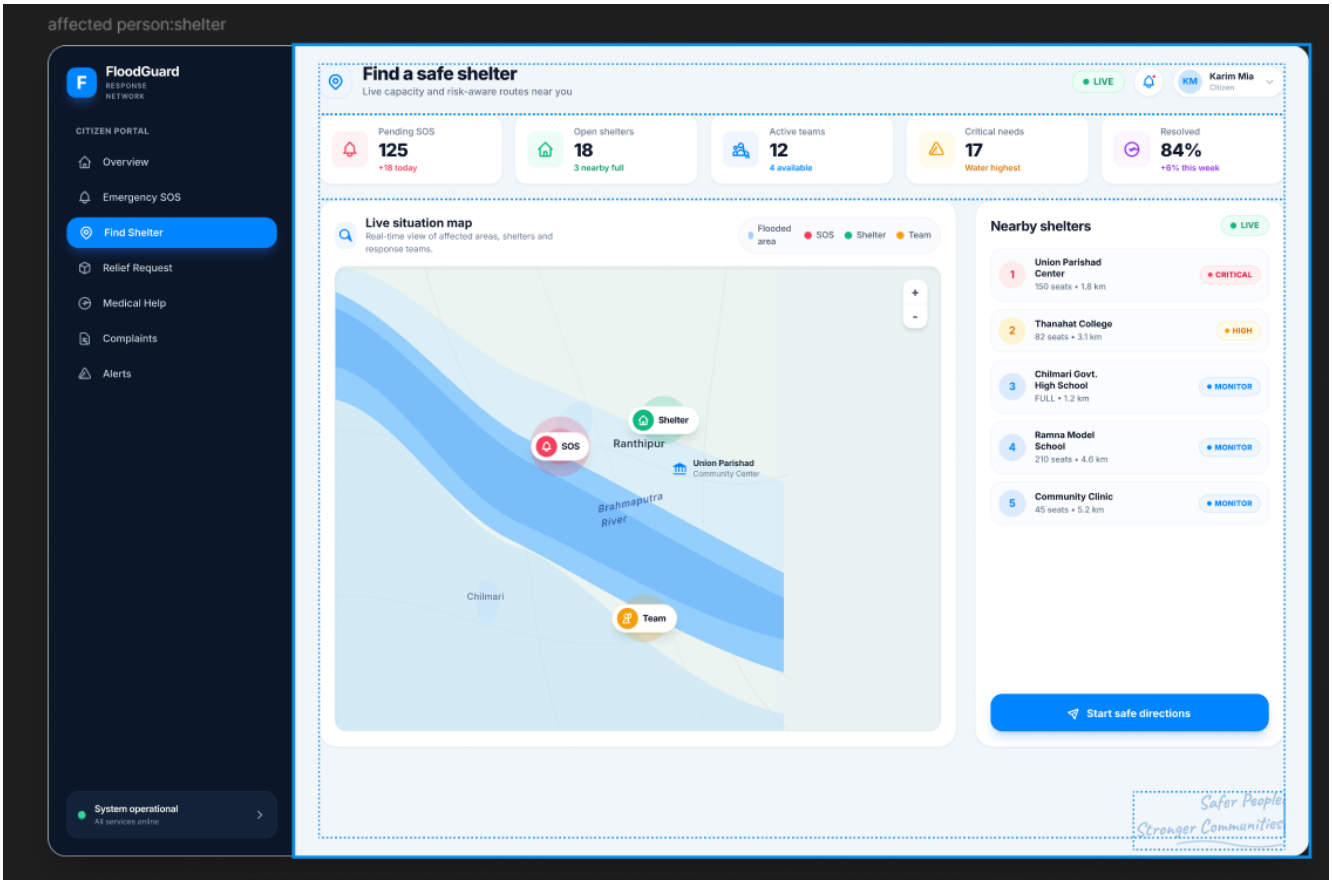


Figure 11.9: Find Safe Shelter prototype

11.10 Relief Request

The relief request screen captures delivery area, household size, requested items, medical support, vulnerable conditions, notes, area-demand context, and live map information.

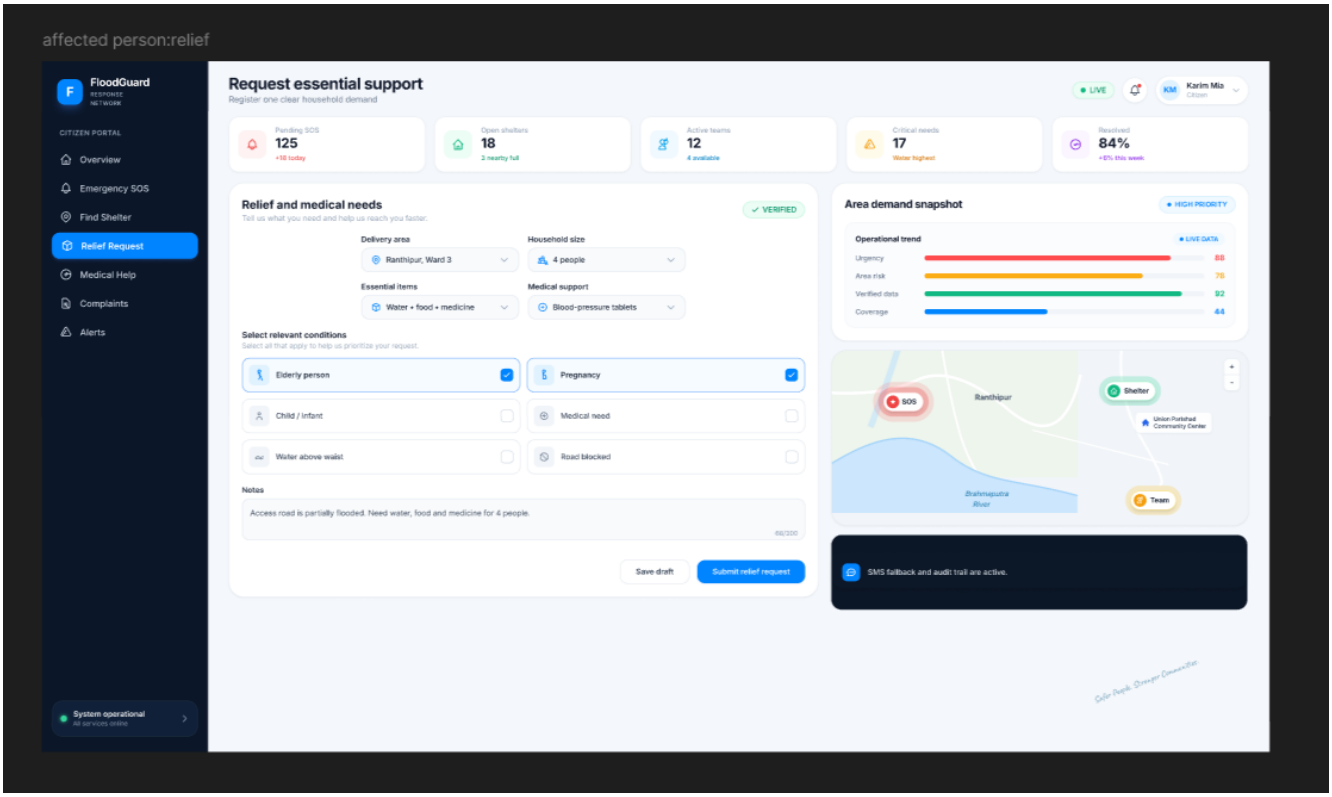


Figure 11.10: Relief Request prototype

11.11 Emergency SOS Request

The SOS form captures GPS location, number of people needing rescue, water level, contact number, special risk conditions, notes, priority assessment, and nearby flood/shelter/team context.

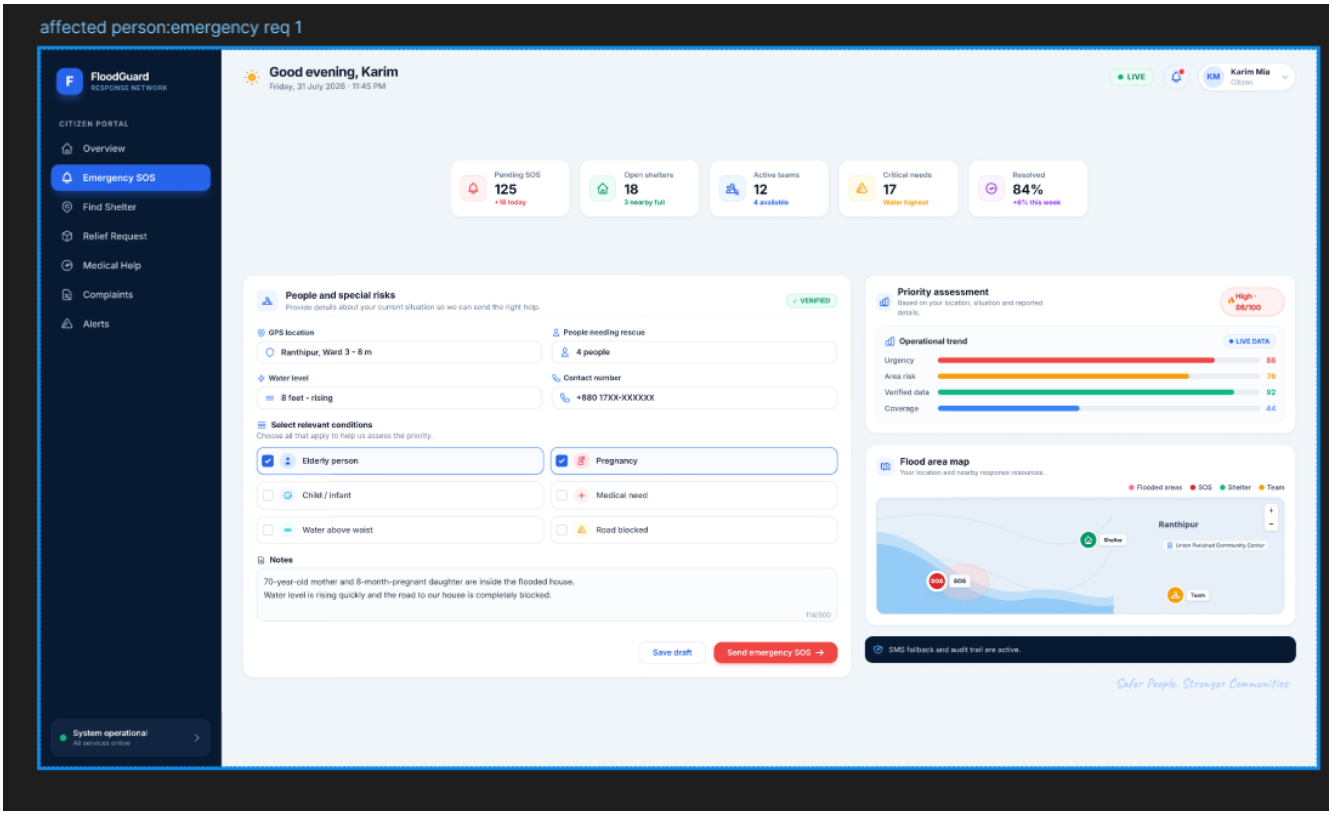


Figure 11.11: Emergency SOS Request prototype

11.12 Volunteer Field Desk

The volunteer overview shows assigned-area status, shelter capacity warning, live map, priority task queue, KPI cards, and recent field activity.

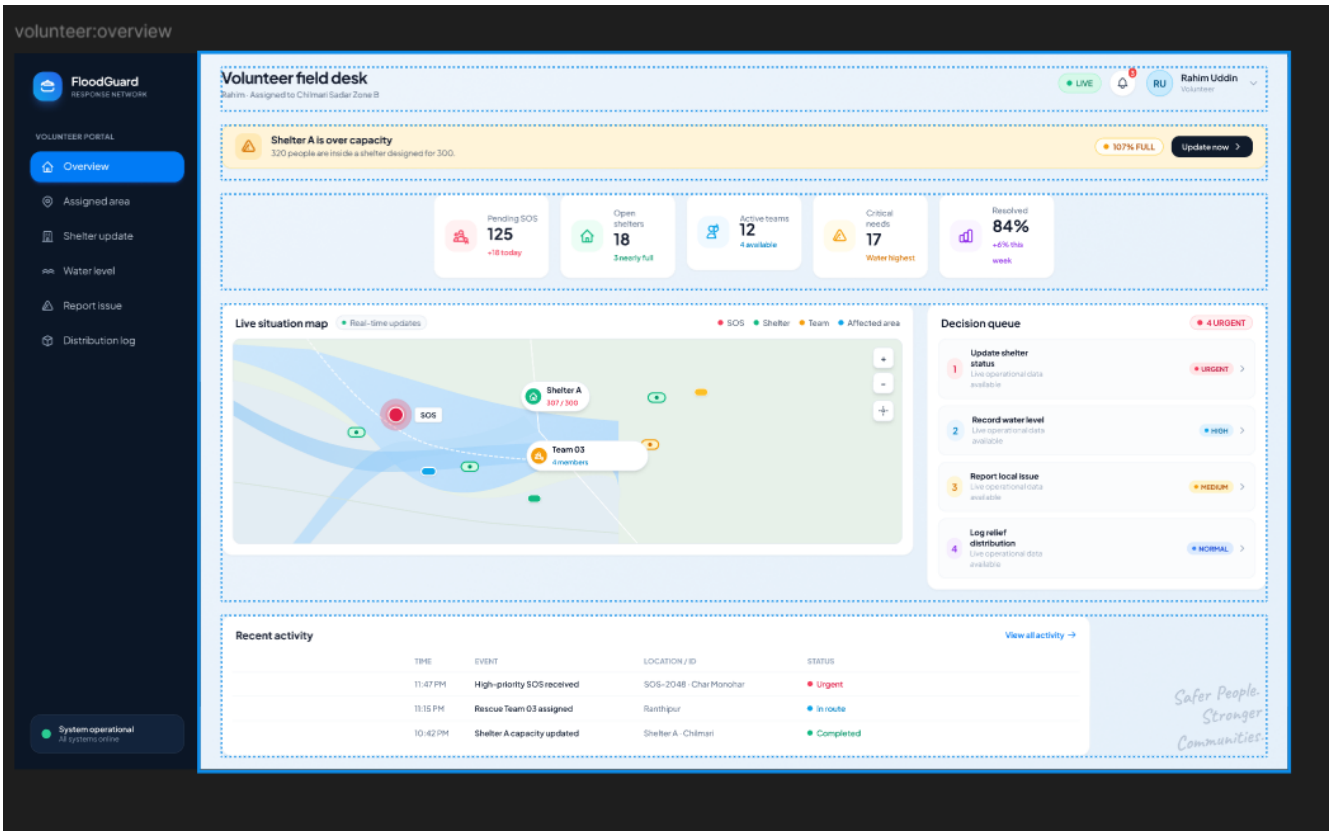


Figure 11.12: Volunteer Field Desk prototype

11.13 Rescue Team Dashboard

The rescue portal focuses on pending SOS, team assignments, completed missions, rescued people, active teams, live map, decision queue, and mission history.

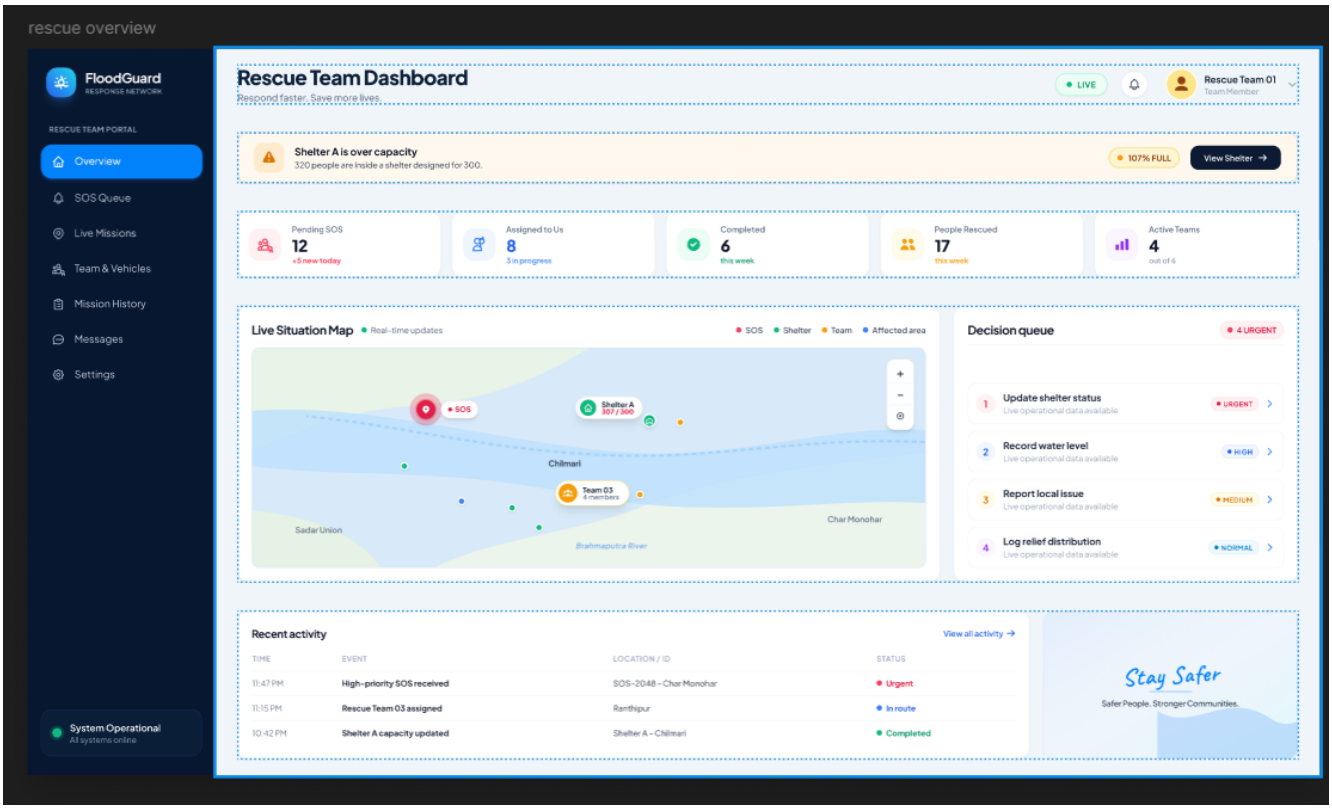


Figure 11.13: Rescue Team Dashboard prototype

11.14 NGO Operations Overview

The NGO portal combines area demand, distribution, inventory and stock, donations, volunteers, grievances, reports, immediate actions, and a resource summary.

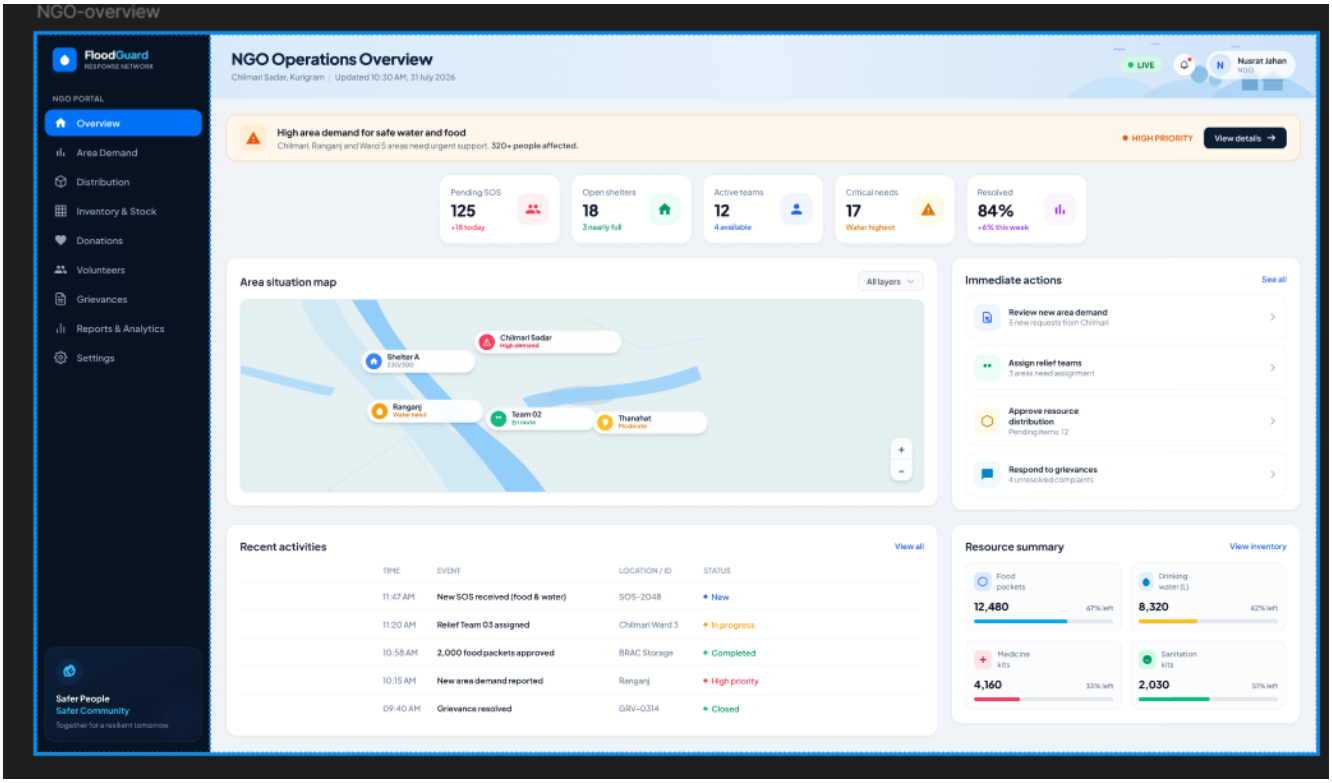


Figure 11.14: NGO Operations Overview prototype

11.15 Donor Command Center

The donor portal visualizes where help is needed, recent donation pledges, donation utilization, impact metrics, and quick actions for creating new pledges or viewing area needs.

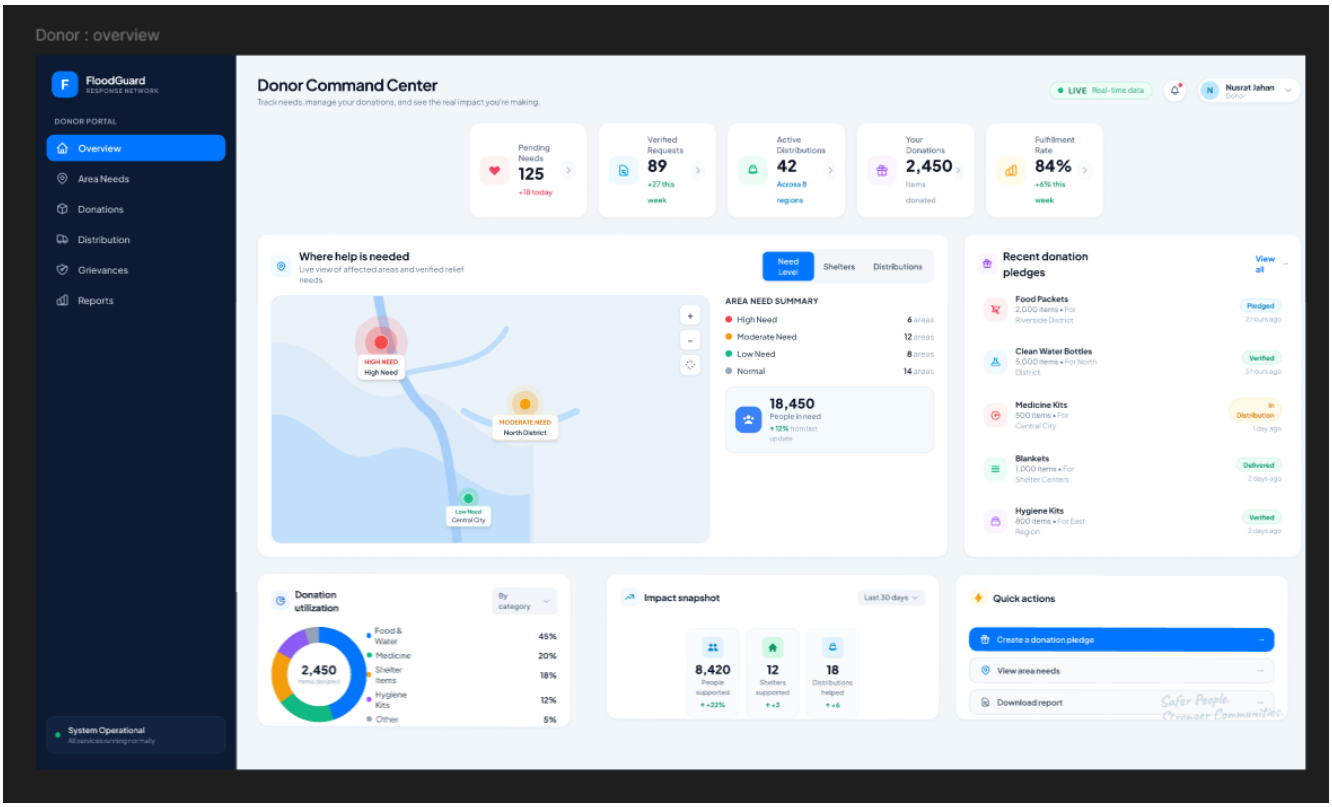


Figure 11.15: Donor Command Center prototype

11.16 Admin Command Center

The administrative command center presents severe incident status, global KPIs, live situation map, prioritized decision queue, broadcast access, activity log, and links to SOS, resources, shelters, teams, stock, users and analytics.

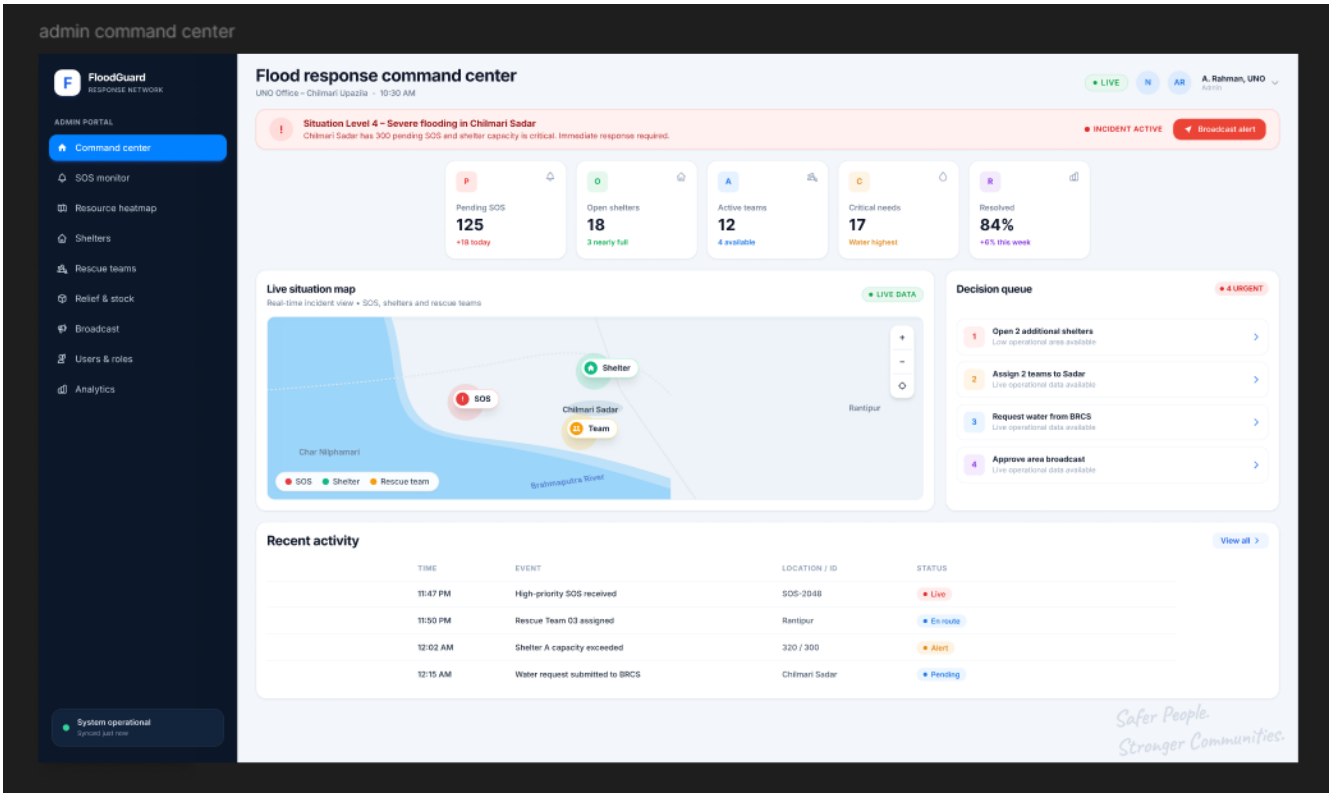


Figure 11.16: Admin Command Center prototype

11.17 Sitemap and Navigation Flow

The sitemap organizes the system around role-based portals. After authentication, users are redirected to the portal associated with their role. Each portal exposes a focused navigation hierarchy while preserving common interaction patterns.

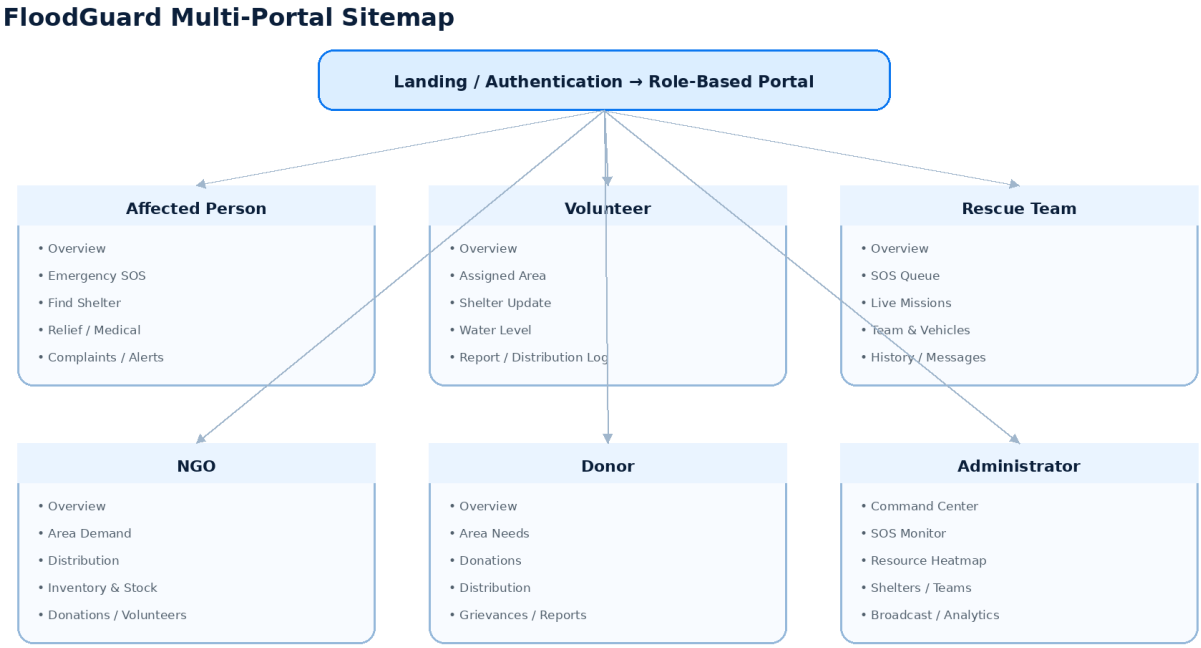


Figure 11.12: FloodGuard multi-portal sitemap

CHAPTER 12: System Architecture

12.1 Holistic Project View

The holistic project view connects stakeholder actions to the frontend/access layer, application services, data and AI integration, and decision outputs. It shows how SOS, rescue, shelter, relief, donations, volunteers, water level, complaints, notifications and reporting participate in one coordinated platform.

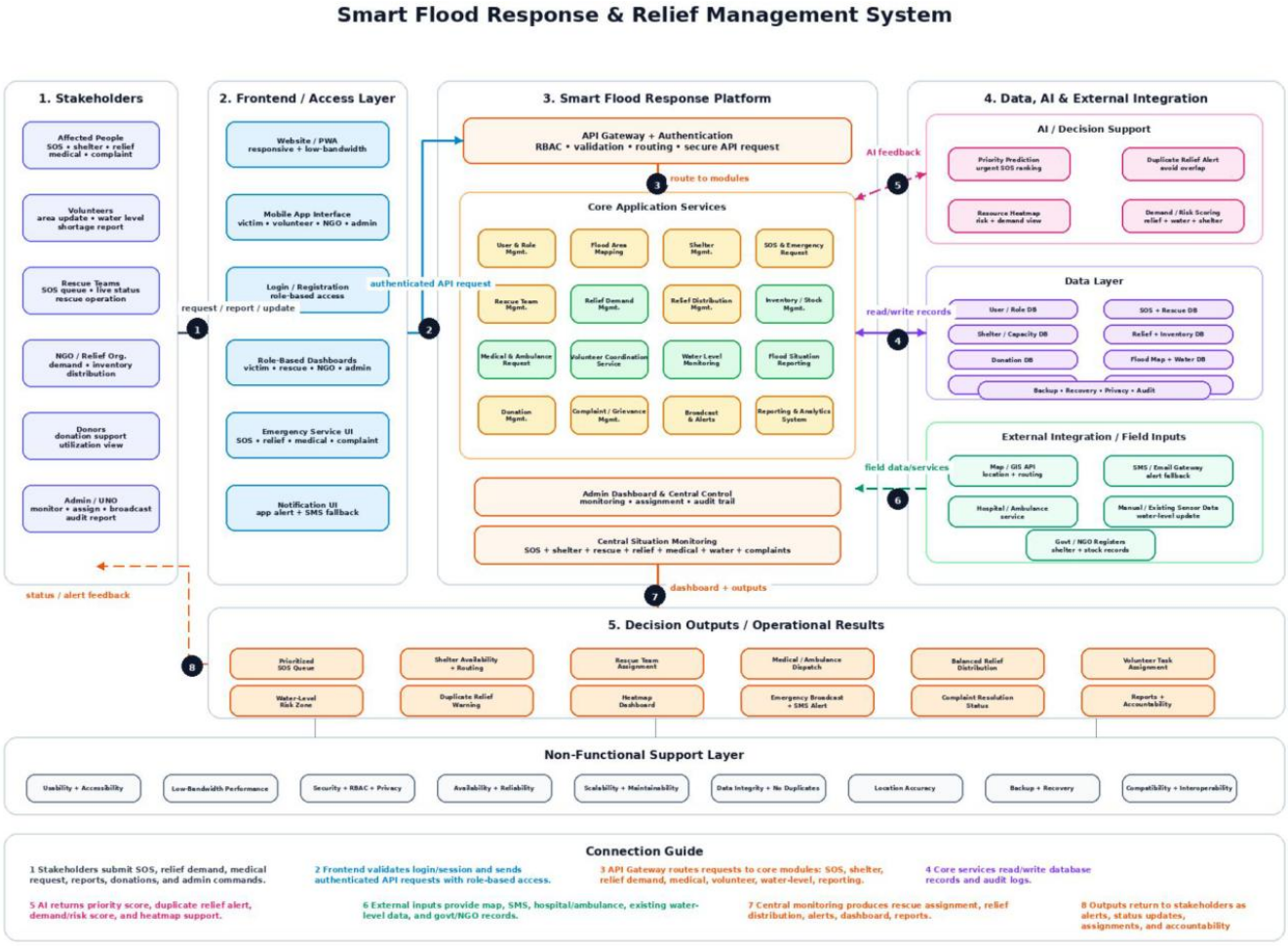


Figure 12.1: Holistic project view

12.2 Infrastructure Architecture

The infrastructure view describes how FloodGuard can run in a real cloud/server environment with edge security, containerized microservices, multiple data stores, event streaming, analytics, observability, CI/CD, multi-zone deployment and disaster recovery.

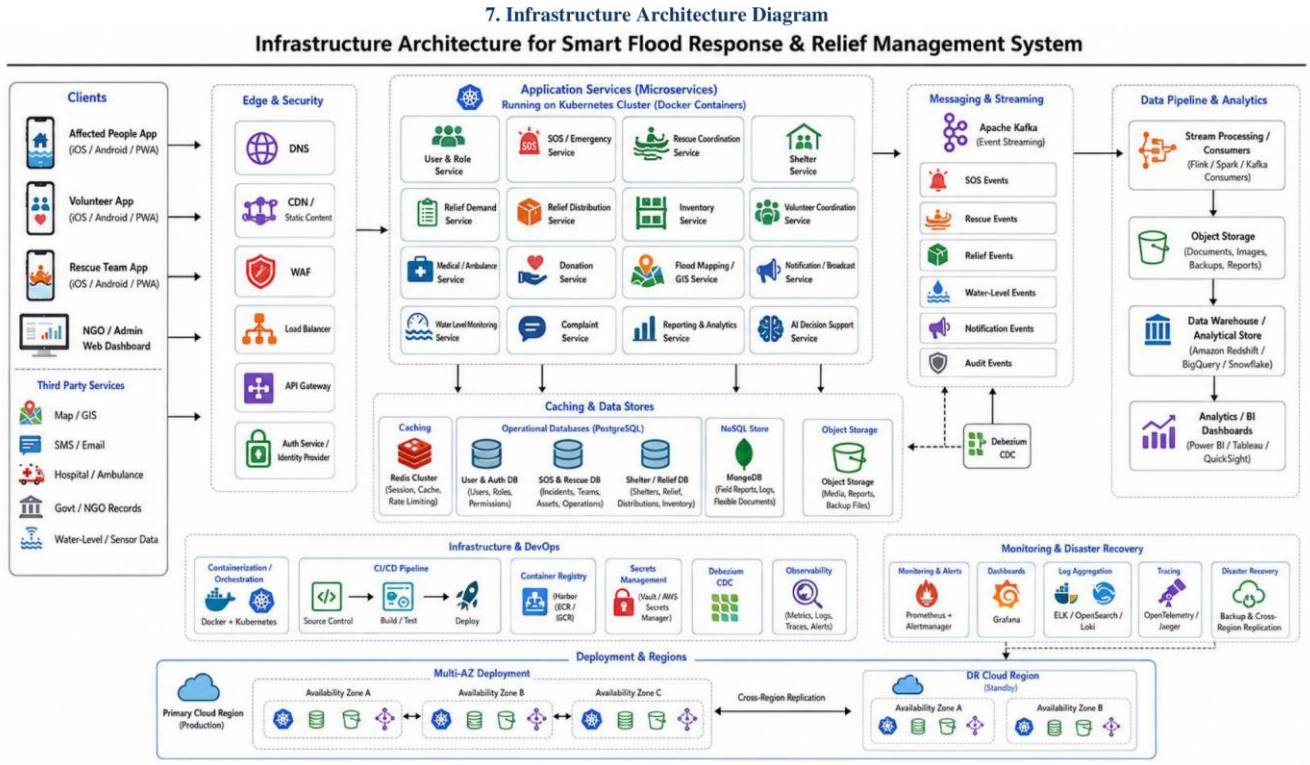


Figure: Infrastructure view of the Smart Flood Response & Relief Management System

Figure 12.2: Infrastructure architecture

12.3 Architecture Flow

- Clients: affected people, volunteers, rescue teams, NGOs, donors and administrators use PWA/mobile/web interfaces; third-party GIS, messaging, hospital, government and water-level services can connect.
- Edge and Security: requests pass through DNS, CDN, WAF, load balancer, API gateway and authentication services.
- Application Services: user, SOS, rescue, shelter, relief demand/distribution, inventory, volunteer, medical/ambulance, donation, flood mapping, notification, water level, complaint, reporting and AI decision support.
- Data Stores: Redis for fast cache/session/queue data; PostgreSQL for structured operational records; MongoDB for flexible field/complaint documents; object storage for images, documents and backups.
- Messaging and Analytics: Kafka carries real-time events; Debezium captures database changes for live dashboards and analytics.
- DevOps, Monitoring and Recovery: Docker/Kubernetes, CI/CD, metrics/logs/tracing, Multi-AZ and DR support maintainability, observability and continuity.

12.4 Technology Justification

Technology	Purpose in FloodGuard
Load Balancer	Distributes incoming emergency traffic and prevents one service instance from becoming overloaded.
API Gateway	Provides one controlled entry point for login, SOS, shelter, relief, donation, report and other APIs.

Containerization / Kubernetes	Runs modular services consistently and supports scaling, rollout and restart.
Redis	Stores fast-changing cache, sessions, rate-limit information and urgent queue data.
PostgreSQL	Stores structured operational data with strong relationships and transaction integrity.
MongoDB	Stores flexible field reports, flood notes, complaint details and other document-style data.
Object Storage	Stores damage photos, documents, generated reports, proof images and backups.
Apache Kafka	Transfers real-time SOS, rescue, relief, water-level, notification and audit events.
Debezium	Captures database changes and publishes them to event streams for dashboards and analytics.
Observability	Metrics, logs, traces and alerts reveal slow APIs, service failures and infrastructure pressure.
Multi-AZ and DR	Supports availability and recovery if a zone or primary environment fails.

CHAPTER 13: Conclusion

FloodGuard is designed as an integrated flood-response coordination system rather than a single-purpose warning, mapping, or logistics tool. The final SRS shows how the proposed platform connects citizens, volunteers, rescue teams, NGOs, donors, and administrators through shared workflows for SOS, rescue, shelter, relief, medical support, inventory, donations, water-level reporting, complaints, broadcasts, and analytics. Benchmarking shows that established platforms provide valuable strengths in humanitarian logistics, crowdsourced mapping, and institutional information sharing; the project applies those lessons to a focused Bangladesh flood-response workflow and adds project-specific decision support. The UML, ER design, UI prototype, and infrastructure architecture provide a consistent blueprint for future implementation and testing.

CHAPTER 14: Future Scope

- Integrate official water-level and forecast feeds through approved APIs or data-sharing arrangements.
- Support offline-first form capture and later synchronization in low-connectivity areas.
- Add verified SMS/USSD emergency flows for users without smartphones.
- Integrate authorized GPS tracking for rescue vehicles and field teams.
- Validate AI priority models with real emergency data, explainable factors, fairness checks, and human override.
- Add route optimization using flood depth, road closure and shelter capacity information.
- Support interoperability with government and humanitarian information systems.
- Add Bangla-first multilingual support, voice guidance, and stronger accessibility features.
- Extend the platform to post-disaster damage assessment, rehabilitation and recovery tracking.
- Conduct field usability testing, load testing, penetration testing, disaster-recovery exercises, and pilot deployment before real-world use.

CHAPTER 15: References

[1] Project Team, “Smart Flood Response & Relief Management System,” internal SRS and system/infrastructure architecture document, 2026.

[2] Project Team, “Entity-Relationship Analysis and ER Diagram - Smart Flood Response & Relief Management System,” 2026.

[3] Group 7, “Software Requirements Specification (SRS) - Initial Proposal,” CSE 3411 Assignment 01, United International University, 2026.

[4] Sahana Software Foundation, “Eden Legacy Archive,” official product page, https://sahanafoundation.org/products/eden/ (accessed 28 Aug. 2026).

[5] Sahana Software Foundation, “Products,” official overview, https://sahanafoundation.org/products/ (accessed 28 Aug. 2026).

[6] Ushahidi, “Features - Inform Decisions & Empower Communities,” official platform features, https://www.ushahidi.com/features/ (accessed 28 Aug. 2026).

[7] International Federation of Red Cross and Red Crescent Societies (IFRC), “IFRC GO,” official emergency operations platform overview, https://www.ifrc.org/happening-now/emergencies/ifrc-go (accessed 28 Aug. 2026).

[8] IFRC GO, “Resources,” official platform resources and coordination tools, https://go.ifrc.org/resources (accessed 28 Aug. 2026).

[9] Project requirements elicitation user-feedback survey (Google Form), 30 respondents, 2026.

15.1 Source Use Note

In the benchmark comparison, “Not indicated” means that the reviewed official feature material did not identify the capability as a core feature. It does not mean that customization or integration is technically impossible. The three benchmarks were selected because they represent complementary strengths: Sahana Eden for humanitarian logistics and shelters, Ushahidi for crowdsourced reporting and mapping, and IFRC GO for institutional emergency information sharing and dashboards.